FinBank 완전 가이드

수신·여신·배치부터 불변식 하네스·AI 에이전트 하네스까지 — 설계·코드·DB·테스트 전부 한 권으로

2026-11-05 개정판 (초판 2026-07-28)

목차

0. 이 문서를 읽는 법 **4**
0-1. 이번 개정에서 바뀐 것 (2026-11-05) . . . 4

1. FinBank가 뭔가요? **5**
1-1. 왜 은행인가? . . . 5
1-2. 무엇으로 만들었나 (기술 목록) . . . 5

2. 은행 업무를 둘로 나누기: 수신과 여신 **6**
2-1. 수신(受信) — 맡아주는 돈 . . . 6
2-2. 여신(與信) — 빌려주는 돈 . . . 6
2-3. 이 프로젝트가 걸어온 순서 . . . 6

3. 데이터베이스 전략 — 표를 어떻게 만들었나 **7**
3-1. 데이터베이스가 뭔가요? . . . 7
3-2. 전체 표 목록 (10개) . . . 7
3-3. 표들이 어떻게 연결되나 . . . 7
3-4. 설계에서 가장 중요한 결정 4가지 . . . 8
3-5. Flyway — DB 변경 이력 관리 . . . 8

4. 패키지(폴더) 구성 — 코드를 어디에 두었나 **10**
4-1. 전체 구조 . . . 10
4-2. 왜 이렇게 나눴나 — “도메인형” 구조 . . . 11
4-3. 의존 방향 — 화살표는 한쪽으로만 . . . 11

5. 수신 파트 — 맡아주는 돈 **12**
5-1. 로그인은 어떻게 되나 (JWT) . . . 12
5-2. 계좌 만들기 . . . 12
5-3. 입금·출금·이체 . . . 12
5-4. 동시성 — 이 프로젝트의 심장 . . . 13

6. 돈을 다루는 규칙 — BigDecimal 이야기 **15**
6-1. 왜 소수가 필요한가 . . . 15
6-2. 해결: BigDecimal + DECIMAL(19,4) . . . 15
6-3. 전환 시점을 왜 그때로 잡았나 . . . 15
6-4. 규칙을 한 곳에 모았다 — `MoneyPolicy` . . . 15

7. 여신 파트 — 빌려주는 돈 **16**
7-1. 대출 상품 (`loan_products`) . . . 16
7-2. 상환 방식 3가지 — 그림으로 이해하기 . . . 16
7-3. 끝수 조정 — 1원이 남는 문제 . . . 17
7-4. 검증을 4층으로 했다 . . . 17
7-5. 자동심사 — 룰을 클래스 하나씩 . . . 17

7-6. 대출 실행 — 돈이 실제로 나가는 순간 . . . 18
7-7. 상환 — 갚기 . . . 18

8. 멱등키 — "두 번 눌러도 한 번만" 19
8-1. 문제 상황 . . . 19
8-2. 해결: 요청마다 고유 번호표 . . . 19
8-3. 화면에서 키를 어떻게 다뤄야 하나 . . . 19
8-4. 트랜잭션 경계 — 가장 어려운 부분 . . . 19
8-5. 경쟁의 심판은 DB에게 맡긴다 . . . 19
8-6. 실제 측정 결과 . . . 20
8-7. 이중 처리 방어 — 층마다 다른 걸 막는다 . . . 20
8-8. 대출 상환의 락 순서 . . . 20
8-9. 이체로 확장 — 같은 장치, 같은 원칙 (PR #19) . . . 21

9. 검색과 인덱스 — 느린 걸 빠르게 22
9-1. 문제: 조건이 7개인 검색 . . . 22
9-2. 해결: QueryDSL . . . 22
9-3. 인덱스 — 책의 목차 . . . 22
9-4. 실제로 재봤다 (10만 건) . . . 22
9-5. 인덱스 컬럼 순서가 중요하다 . . . 22
9-6. 정직하게 기록한 한계 . . . 22

10. 배치 — 밤에 혼자 도는 프로그램 23
10-1. 배치가 뭔가요? . . . 23
10-2. Spring Batch의 뼈대 — Job · Step · 청크 . . . 23
10-3. 스케줄러 — 언제, 무엇을, 어떤 순서로 . . . 24
10-4. 연체 감지 — 상태를 바꾸고 이력을 연다 . . . 25
10-5. 가장 조심해야 할 함정: 페이징 밀림 . . . 25
10-6. 연체이자 가산 — 밀린 날만큼, 겹치지 않게 . . . 26
10-7. 멱등을 세 겹으로 . . . 27
10-8. 상환과 맞물리는 지점 — 연체 해소 . . . 27
10-9. 청크 사이즈 — 재보고 정했다 . . . 27
10-10. 운영 관점 — 놓쳤을 때, 실패했을 때, 다시 돌릴 때 . . . 28
10-11. 스케줄러의 한계를 정직하게 적었다 . . . 28
10-12. 테스트 12개가 지키는 것 . . . 28

11. 예외 처리 — 오류를 어떻게 알려주나 30
11-1. 상태 코드로 구분한다 . . . 30
11-2. ErrorCode로 한 곳에 모았다 . . . 30
11-3. 점진 이관 (expand-contract) . . . 30
11-4. 알고 있는 빈틈 — 엔진의 거부가 API까지 오면 . . . 30

12. 테스트 — 508개가 무엇을 지키나 32
12-1. 철학: 개수보다 "위험한 곳" . . . 32
12-2. 테스트 파일별 역할 (전체) . . . 32
12-3. 왜 진짜 MySQL로 테스트하나 . . . 34
12-4. 성능 테스트를 분리한 이유 . . . 34
12-5. 테스트 DB 분리 . . . 34

13. 불변식 하네스 — 정답을 몰라도 틀린 걸 잡는 법 35
13-1. 문제: 정답을 누가 정하나 . . . 35
13-2. 불변식 — 입력이 무엇이든 항상 참인 규칙 . . . 35
13-3. 판정기 — JUnit 없이, 위반을 전부 모은다 . . . 36
13-4. 입력은 네 군데서 온다 . . . 36
13-5. 무작위 탐색이 찾은 진짜 버그 . . . 37
13-6. 입력 계약 — 엔진이 받아들일 입력의 경계 . . . 38
13-7. 이 하네스가 다루지 않는 것 . . . 38

14. AI 에이전트 하네스 — 만드는 AI와 검증하는 AI를 나누다 **39**
14-1. 왜 이걸 했나 — “AI가 만든 결과를 어떻게 믿나” . . . 39
14-2. 원칙 세 가지 . . . 39
14-3. 누가 무엇을 할 수 있나 . . . 39
14-4. 역할을 주는 방법 — md 파일 세 종류 . . . 40
14-5. 통제하는 방법 — 강제 장치 3겹 . . . 41
14-6. 가드도 테스트한다 . . . 42
14-7. 실전 기록 — 두 터미널로 돌려 보다 (2026-11-04) . . . 43
14-8. 한 라운드의 흐름 . . . 43
14-9. 지금 상태와 한계 (2026-11-05) . . . 44

15. 자동화와 실행 환경 **45**
15-1. CI (GitHub Actions) — 자동 감독관 . . . 45
15-2. 판정 기준 가드 (Harness Guard) — 기준이 바뀌면 사람이 승인 . . . 45
15-3. Docker Compose — 한 번에 켜기 . . . 45
15-4. 설정 우선순위 . . . 45

16. 실패 기록 12건 — 무엇이 왜 깨졌나 **46**
① BigDecimal 전환 후 테스트 전멸 . . . 46
② Flyway 유령 마이그레이션 . . . 46
③ 코드를 안 고쳤는데 어느 날 깨지는 테스트 . . . 46
④ 인덱스를 지울 수 없다 (MySQL 에러 1553) . . . 46
⑤ 배치 테스트가 두 번째부터 실패 . . . 46
⑥ 배치를 만들자 드러난 상환의 빈틈 . . . 46
⑦ PAT 권한 부족으로 push 거부 . . . 46
⑧ 테스트 설정 파일 하나 추가했더니 컨텍스트 전체가 죽었다 . . . 47
⑨ 240조합이 놓친 원리금균등의 음수 잔액 (2026.10, PR #20) . . . 47
⑩ 상환에는 있고 이체에는 없던 멱등키 (2026.10, PR #19) . . . 47
⑪ diff 가드가 에이전트를 범인으로 지목했다 (.DS_Store) . . . 47
⑫ 에이전트를 지켜보다 찾은 하네스의 구멍 . . . 47

17. 면접 예상 질문과 답 **48**

18. 한 장 요약 **51**
이 프로젝트가 증명하는 것 . . . 51
전 구간 흐름 . . . 51
개발 흐름 — AI 에이전트 하네스 . . . 51
처음부터 다시 만든다면 — 순서 . . . 51

부록 A. 코드 상세 레퍼런스 **53**
A-1. 도메인 — 자기 규칙을 스스로 지키는 객체 . . . 53
A-2. 계산 엔진 — 스프링을 모르는 순수 자바 . . . 54
A-3. 리포지토리 — DB에 말 거는 층 . . . 55
A-4. 서비스 — 규칙이 사는 곳 . . . 56
A-5. 멱등 실행기 — 전체 코드 . . . 57
A-6. 컨트롤러 — 엔드포인트 전체 목록 . . . 58
A-7. 프론트엔드에서 멱등키 다루기 — 공통 모듈 하나로 . . . 59
A-8. 하네스 파일 레퍼런스 . . . 59

부록 B. 용어 사전 **61**

0. 이 문서를 읽는 법

이 문서는 **FinBank라는 프로젝트를 하나도 모르는 사람**이 처음부터 끝까지 읽으면 "아, 이 사람이 뭘 만들었고 왜 그렇게 만들었구나"를 알 수 있게 쓴 글입니다.

어려운 말이 나오면 반드시 그 자리에서 쉬운 말로 다시 설명합니다. 순서는 이렇습니다.

1. **FinBank가 뭔지** — 한 문장으로
2. **은행 업무를 둘로 나누기** — 수신(맡는 돈)과 여신(빌려주는 돈)
3. **데이터베이스 전략** — 표(테이블)를 어떻게 만들었고 왜 그렇게 만들었나
4. **폴더(패키지) 구성** — 코드를 어디에 두었나
5. **수신 파트** — 회원가입·로그인·계좌·입출금·이체, 그리고 **동시성**
6. **돈을 다루는 규칙** — 소수점과 반올림 이야기
7. **여신 파트** — 대출 상품·신청·심사·이자계산·실행·상환
8. **멱등키** — "두 번 눌러도 한 번만"
9. **검색과 인덱스** — 느린 걸 빠르게
10. **배치** — 밤에 혼자 도는 프로그램 (구조·코드·운영까지)
11. **예외 처리** — 오류를 어떻게 알려주나
12. **테스트** — 508개가 무엇을 지키고 있나
13. **불변식 하네스** — 정답을 몰라도 틀린 걸 잡는 법
14. **AI 에이전트 하네스** — 만드는 AI와 검증하는 AI를 나누다
15. **자동화와 실행 환경** — CI, Docker, 판정 기준 가드
16. **실패 기록 12건** — 무엇이 왜 깨졌고 어떻게 고쳤나
17. **면접 예상 질문과 답**
18. **한 장 요약**

부록 A는 실제 코드를 계층별로, 부록 B는 용어 사전입니다.

0-1. 이번 개정에서 바뀐 것 (2026-11-05)

초판(2026-07-28) 이후에 한 일을 더하고, 초판에서 실제 코드와 어긋나 있던 곳을 바로잡았습니다.

새로 쓴 것

위치	내용
5-3, 8-9	이체 멱등키 · 락을 걸기 전에 권한 확인 · 잠금 계좌 입출금 차단 (PR #19)
7-2, 7-4	원리금균등이 깨지는 조합(만기 전 잔액 음수)을 찾아 거부하도록 수정, 검증 4층 (PR #20)
10장	배치를 다시 씀 — Job·Step·Reader·Processor·Writer 구조와 실제 코드, 상환과 맞물리는 지점, 놓쳤을 때·실패했을 때, 테스트 12개
11-4	알고 있는 빈틈 — 엔진의 거부 예외가 API까지 오면 500
13장	불변식 하네스 — 불변식 카탈로그 25개, 판정기, 케이스 파일, 무작위 탐색, 입력 계약
14장	AI 에이전트 하네스 — 구현·검증 역할 분리, hook·diff 가드·CI 가드, 두 터미널 실전 기록
15-2	판정 기준 가드 CI (Harness Guard)
16장 ⑨~⑫	실패 기록 4건
17장 Q9~Q15	배치 운영·하네스 관련 질문
부록 A-8	하네스 파일 레퍼런스

바로잡은 것

위치	초판	지금 (실제 코드 기준)
0장 목차	예외 처리 장이 빠져 11장부터 번호가 어긋남	장 번호와 일치
3-5, 18장	마이그레이션 V1~V6	V1~V7 (V7: 대출 신청 시 입금 계좌 지정, 2026-08)

위치	초판	지금 (실제 코드 기준)
4-1, 12-2, 18장	자동심사 룰 3개	2개 — `HasActiveAccountRule`은 V7 때 도달 불가능해져 삭제 (7-5)
5-4	락 없는 쪽 결과를 고정값처럼 적음	실행할 때마다 달라지는 값임을 명시
12장, 15장, 18장	테스트 333개	508개 (성능 측정 2개는 별도)
12-2	`LoanApplicationServiceTest` 8개 · `TransferRuleTest` 6개 · `ScheduleGeneratorTest` 12개	11개 · 10개 · 14개
부록 A-3	연체 후보 조회 쿼리를 단순화해 적음	실제 쿼리 (`join fetch` 포함)
부록 A-6	멱등키 필수 API 3개	4개 (이체 추가)
부록 A-7	화면마다 키를 다루는 예시	공통 모듈 `api/idempotency.js`

---

1. FinBank가 뭔가요?

FinBank는 "은행 서버"를 흉내 내서 만든 개인 프로젝트입니다.

화면(웹사이트)도 있지만, 진짜 핵심은 **눈에 보이지 않는 뒤쪽(백엔드)**입니다. 돈을 넣고, 빼고, 옮기고, 빌려주고, 갚는 모든 일이 여기서 벌어집니다.

1-1. 왜 은행인가?

프로그램을 만들 때 가장 어려운 문제 중 하나가 **"여러 사람이 같은 것을 동시에 만졌을 때 값이 틀어지는 것"**입니다. 그런데 은행은 이게 틀어지면 **바로 돈이 사라지거나 생깁니다.** 그래서 이 문제를 공부하기에 가장 좋은 소재입니다.

> **12살 버전**: 친구 다섯 명이 동시에 같은 저금통에 손을 넣어서 돈을 꺼낸다고 생각해 보세요. 저금통에 3천 원이 있는데 다섯 명이 각각 "1천 원 꺼낼게!"라고 동시에 외치면? 잘못 만들면 5천 원이 나와 버립니다. **없던 돈이 생기는 것**이죠. FinBank는 이런 일이 절대 일어나지 않게 막는 방법을 연구한 프로젝트입니다.

1-2. 무엇으로 만들었나 (기술 목록)

무엇	이름	쉬운 설명
언어	Java 17	프로그램을 쓰는 말
뼈대	Spring Boot 3	서버를 쉽게 만들게 도와주는 도구상자
데이터 저장소	MySQL 8	표(엑셀 비슷한 것)로 자료를 저장하는 창고
자료 다루기	JPA / Hibernate	자바 객체와 DB 표를 자동으로 연결해 주는 통역사
표 관리	Flyway	DB 표의 변경 이력을 순서대로 관리하는 공책
검색	QueryDSL	조건이 여러 개인 검색을 자바 코드로 안전하게 쓰는 도구
밤 작업	Spring Batch	대량 작업을 안전하게 돌리는 도구
로그인	JWT + Spring Security	출입증을 만들고 검사하는 장치
화면	React + Vite + Tailwind	사용자가 보는 웹 화면
자동 검사	GitHub Actions	코드를 올릴 때마다 자동으로 시험을 보는 감독관
실행 환경	Docker Compose	서버와 DB를 한 번에 켜는 스위치
AI 에이전트	Claude Code	코드를 만드는 AI와 검증하는 AI를 따로 띄우는 도구
판정 기준 가드	Python 3 · Bash	AI가 고치면 안 되는 파일을 막는 문지기

마지막 두 줄은 2026년 10~11월에 더한 **하네스** 부분입니다(13·14장). 서비스 기능이 아니라 **개발 과정**에 들어가는 도구라, 실행 중인 FinBank 서버에는 AI가 들어 있지 않습니다.

---

2. 은행 업무를 둘로 나누기: 수신과 여신

은행이 하는 일은 크게 **두 가지**입니다. 이 프로젝트도 정확히 그 둘로 나뉩니다.

2-1. 수신(受信) — 맡아주는 돈

"손님이 맡긴 돈을 보관하고 옮겨주는 일"입니다.

- 통장 만들기 (계좌 개설)
- 돈 넣기 (입금)
- 돈 빼기 (출금)
- 다른 통장으로 보내기 (이체)

  12살 버전: 저금통을 만들어 주고, 돈을 넣고 빼주고, 친구 저금통으로 옮겨주는 일이에요.

2-2. 여신(與信) — 빌려주는 돈

"손님에게 돈을 빌려주고 이자를 받아 돌려받는 일"입니다.

- 대출 상품 (어떤 조건으로 빌려주는지)
- 대출 신청과 심사 (빌려줘도 되는 사람인가?)
- 대출 실행 (실제로 돈을 줌)
- 상환 (매달 나눠서 갚음)
- 연체 (안 갚으면 이자가 더 붙음)

  12살 버전: 친구에게 만 원을 빌려주고 "매달 천 원씩 열 달 동안 갚아. 대신 조금 더 얹어서 줘"라고 하는 거예요. 그 '조금 더'가 **이자**입니다.

2-3. 이 프로젝트가 걸어온 순서

```
A. 동시성 문제 증명        (돈이 증발하는 걸 눈으로 보여주기)
B. 금액 타입 전환          (정수 → 소수 가능한 타입으로)
C. 여신 만들기             (상품 → 신청/심사 → 이자엔진 → 실행/상환 → 검색)
D. 배치                    (밤마다 연체를 찾아내고 이자를 붙이기)
E. 마감                    (자동 검사, Docker, 문서 정리)
──────────────────────────────── 2026.10 ~ 11 ────────────────────────────────
F. 정합성 보강  PR #19     (이체 멱등키, 락 전 권한 확인, 잠금 계좌 입출금 차단)
G. 불변식 하네스  PR #20   (정답 없이 판정하는 검증 장치 → 숨은 버그 1건 발견 · 수정)
H. AI 에이전트 하네스      (만드는 AI와 검증하는 AI를 나누고, 판정 기준을 코드로 지킨다)
```

중요한 원칙 세 가지를 처음부터 끝까지 지켰습니다.

1. **깊이 우선** — 기능을 넓게 여러 개 얕게 만들지 않고, 은행의 핵심인 대출 하나를 깊게 팠습니다.
2. **기술은 필요할 때만** — "이거 없으면 뭐가 안 되는데?"에 한 문장으로 답할 수 없으면 안 씁니다. 그래서 QueryDSL은 조건 검색이 생긴 C4에서, Spring Batch는 연체 정산이 생긴 D에서 넣었습니다. Redis 같은 건 **필요를 증명할 수 없어서 일부러 안 넣었습니다.**
3. **AI는 믿을 장치를 만든 다음에** — AI로 기능을 하나 더 붙이는 대신, **AI가 만든 결과를 어떻게 믿을 것인가**를 먼저 풀었습니다. 판정은 사람이 정한 기준(불변식)과 결정적인 코드(테스트·hook·CI)가 하고, AI는 구현과 케이스 만들기만 합니다(G → H 순서인 이유).

---

3. 데이터베이스 전략 — 표를 어떻게 만들었나

3-1. 데이터베이스가 뭔가요?

엑셀 표를 아주 많이, 아주 빠르게, 아주 안전하게 다루는 창고라고 생각하면 됩니다.

- **테이블(표)**: members(회원 표), accounts(계좌 표)처럼 한 종류의 자료를 담는 표
- **행(row)**: 표의 한 줄. 회원 한 명, 계좌 하나
- **컬럼(열)**: 표의 세로 칸. 이름, 잔액, 만든 날짜
- **PK(기본키)**: 그 줄을 콕 집어 가리키는 번호표. 학생 번호 같은 것
- **FK(외래키)**: 다른 표의 번호표를 가리키는 손가락. "이 계좌는 3번 회원 거예요"
- **UNIQUE(유일 제약)**: "이 칸에는 같은 값이 두 번 들어올 수 없다"는 규칙

3-2. 전체 표 목록 (10개)

FinBank의 표는 **수신 4개 + 여신 6개**입니다.

수신 쪽 (4개)

표 이름	무엇을 담나
members	회원 (이메일, 이름, 비밀번호)
refresh_tokens	로그인 연장용 티켓
accounts	계좌 (계좌번호, 잔액, 잠김 여부)
transaction_logs	입출금·이체 **원장**(모든 거래 기록)

여신 쪽 (6개)

표 이름	무엇을 담나
loan_products	대출 상품 (금리, 한도, 상환방식)
loan_applications	대출 신청서 (입금받을 계좌 지정 + 심사 결과 포함)
loan_accounts	실행된 대출 계약
repayment_schedules	상환 스케줄 (1회차, 2회차… 각각 한 줄)
loan_transactions	대출 **원장** (실행/상환/연체이자)
overdue_histories	연체 이력

여기에 **멱등키 표** idempotency_keys 하나와, Spring Batch가 자동으로 만드는 BATCH_* 표들이 더 있습니다.

3-3. 표들이 어떻게 연결되나

```
members (회원)
   ├──1:1─ refresh_tokens (로그인 티켓)
   ├──1:N─ accounts (계좌) ──1:N── transaction_logs (입출금 원장)
   ├──1:N─ loan_applications (대출 신청)
   │          └──1:1─ loan_accounts (대출 계약)
   │                    ├──1:N─ repayment_schedules (상환 회차)
   │                    │          └──1:N─ overdue_histories (연체 이력)
   │                    └──1:N─ loan_transactions (대출 원장)
   └──1:N─ loan_accounts (직접 연결도 있음 — 조회 편의)

loan_products (상품) ──1:N── loan_applications
accounts (입출금 계좌) ──1:N── loan_applications (입금받을 계좌 지정, V7)
accounts (입출금 계좌) ──1:N── loan_accounts (연결계좌: 실행 시 입금 / 상환 시 출금)
```

> **1:N이 뭔가요?** "하나에 여럿"이라는 뜻입니다. 회원 한 명이 계좌를 여러 개 가질 수 있으니 members 1:N accounts입니다.

3-4. 설계에서 가장 중요한 결정 4가지

① 원장(元帳)은 지우지 않는다 — append-only

`transaction_logs`와 `loan_transactions`는 **한 번 쓰면 절대 수정하거나 지우지 않습니다.** 잘못됐으면 반대 방향 기록을 새로 추가합니다.

> **12살 버전**: 일기장에 잘못 썼다고 찢어내면 안 됩니다. "어제 쓴 건 틀렸어요, 사실은 이랬어요"라고 **새 줄에 적는 것**입니다. 그래야 나중에 무슨 일이 있었는지 전부 알 수 있습니다.

왜? 은행에서 "그때 진짜 얼마가 오갔나"를 나중에 증명할 수 없으면 큰일 나기 때문입니다.

② 거래 직후 잔액을 같이 적는다 — `balance_after`

거래 기록에 "이 거래 후 잔액이 얼마가 됐는지"를 **함께** 저장합니다.

왜? 계좌 표의 잔액이 이상해 보일 때, 원장을 처음부터 더해보면 어디서 틀어졌는지 찾을 수 있기 때문입니다. 검산용 안전장치입니다.

③ 대출 실행 시점의 조건을 "사진 찍어" 보관한다 — 스냅샷

`loan_accounts` 표에는 `interest_rate`(금리), `overdue_extra_rate`(연체가산율), `repayment_type`(상환방식)이 **다시 저장**되어 있습니다. 상품 표에도 있는데 왜 또 저장할까요?

은행이 내일 상품 금리를 바꿔도, 어제 계약한 사람의 조건은 절대 바뀌면 안 되기 때문입니다.

> **12살 버전**: 친구랑 "만 원 빌려주고 매달 100원 이자"라고 약속했는데, 다음 달에 내가 "이제 200원이야"라고 바꾸면 안 되잖아요. **약속한 순간의 조건을 종이에 적어두는 것**이 스냅샷입니다.

④ 규칙은 DB에도 새겨 넣는다 — 제약조건

프로그램 코드로도 막고, **DB 표 자체에도** 막습니다. 이중 잠금입니다.

제약	무엇을 막나
`uq_loan_account_application`	신청 1건은 **딱 한 번만** 실행 가능
`uq_schedule_installment`	같은 대출에 같은 회차 번호가 두 번 생기지 못함
`uq_loan_tx_idem`	같은 멱등키로 거래가 두 번 기록되지 못함
`uq_idem_key`	같은 멱등키가 두 번 선점되지 못함

왜 두 겹인가? 코드는 실수로 빠뜨릴 수 있지만, **DB 제약은 무슨 일이 있어도 뚫리지 않습니다.** 마지막 방어선입니다.

3-5. Flyway — DB 변경 이력 관리

DB 표를 바꿀 때 손으로 직접 고치면, 내 컴퓨터와 서버의 표 모양이 달라져서 사고가 납니다. 그래서 **변경을 SQL 파일로 번호를 매겨 저장**하고, 프로그램이 켜질 때 순서대로 적용합니다.

파일	무엇을 했나
`V1__init_schema.sql`	회원·계좌·거래원장 표 생성
`V2__add_refresh_tokens.sql`	로그인 연장 티켓 표 추가
`V3__money_to_decimal.sql`	**금액 타입을 정수에서 소수 가능 타입으로 전환**
`V4__create_loan_tables.sql`	여신 6개 표 생성
`V5__create_idempotency_keys.sql`	멱등키 표 + 중도상환수수료율 컬럼
`V6__add_search_indexes.sql`	조회 경로 인덱스 2개 (대출 검색용 + 연체 감지 배치용)
`V7__add_application_linked_account.sql`	대출 신청서에 **입금받을 계좌** 컬럼 추가 (2026-08)

두 가지 철칙이 있습니다.

1. **이미 적용된 파일은 절대 수정하지 않는다.** 고칠 게 있으면 새 번호를 만든다. V7이 실제 예입니다 — 신청서에 계좌 컬럼이 필요해졌을 때 이미 적용된 V4를 고치지 않고 새 파일로 추가했습니다. 기존 데이터가 있는 환경에서 실패하지 않도록 **NULL을 허용**하고, NULL인 옛 신청은 실행할 때 종전처럼 회원의 첫 번째 활성 계좌로 넘어가게(폴백) 했습니다.
2. **JPA는 `ddl-auto: validate`.** 즉 JPA가 표를 마음대로 만들거나 바꾸지 못하게 하고, **"코드와 표 모양이 다르면 시작을 거부"**하게만 시킵니다.

   왜 validate인가? update로 두면 JPA가 알아서 표를 바꿔주는데, **무엇이 언제 바뀌었는지 기록이 안 남습니다.** 실무에서는 금지입니다.

---

4. 패키지(폴더) 구성 — 코드를 어디에 두었나

4-1. 전체 구조

```
com.finbank.backend
  ├── account/     수신: 계좌 · 입출금 · 이체
  │    ├── controller/   AccountController
  │    ├── service/      AccountService, AccountTransferService (이체 멱등 껍데기)
  │    ├── domain/       Account, TransactionLog, TransactionType
  │    ├── repository/   AccountRepository, TransactionLogRepository
  │    └── dto/          요청 · 응답 그릇
  │
  ├── auth/        인증: 회원가입 · 로그인 · 토큰
  │    ├── controller/   AuthController
  │    ├── service/      AuthService
  │    ├── security/     JwtTokenProvider, JwtAuthenticationFilter
  │    ├── domain/       RefreshToken
  │    └── dto/
  │
  ├── member/      회원
  │    ├── domain/       Member
  │    ├── repository/   MemberRepository
  │    └── service/      CurrentMemberProvider (로그인한 사람 찾기 — 단일 창구)
  │
  ├── loan/        여신: 이 프로젝트의 핵심
  │    ├── controller/   LoanProductController, LoanApplicationController, LoanController
  │    ├── service/      LoanApplicationService, LoanExecutionService/Processor,
  │    │                 LoanRepaymentService/Processor, LoanQueryService
  │    │    └── review/   자동심사 룰들 (LoanReviewer 인터페이스 + 룰 2개:
  │    │                  CreditLimitRule, NoActiveOverdueRule)
  │    ├── domain/       LoanProduct, LoanApplication, LoanAccount,
  │    │    │            RepaymentSchedule, LoanTransaction, OverdueHistory + enum들
  │    │    └── calculator/  이자 · 수수료 계산 엔진 (순수 자바)
  │    ├── batch/        연체 감지 · 연체이자 가산 배치
  │    ├── repository/   QueryDSL 커스텀 포함
  │    ├── dto/
  │    └── config/       계산기 빈 등록
  │
  ├── common/      공통
  │    ├── MoneyPolicy       돈 계산 규칙의 단일 출처
  │    ├── PageResponse      페이징 응답 그릇
  │    └── idempotency/      멱등키 장치
  │
  ├── config/      SecurityConfig, SwaggerConfig, QuerydslConfig
  └── exception/   ErrorCode, ApiError, 예외 3종, GlobalExceptionHandler
```

저장소 전체로 보면 하네스(13·14장)가 들어오면서 폴더가 몇 개 늘었습니다.

```
FinBank/
  ├── finbank-backend/
  │    ├── src/main/java/...                  위 구조 (구현 에이전트가 고칠 수 있는 유일한 곳)
  │    ├── src/main/resources/db/migration/   V1~V7
  │    ├── src/test/java/.../loan/harness/    불변식 하네스 (판정 기준)
  │    └── src/test/resources/harness/        케이스 파일 2개 (사람용 / AI용)
  ├── finbank-frontend/                       React + Vite
  ├── scripts/harness/                        하네스 가드 (Python · Bash)
  ├── .claude/                                AI 에이전트 설정 (역할 정의 · hook 등록)
  ├── harness-reports/                        에이전트 작업 기록 (작업 명세 · 검증 리포트)
  ├── .github/workflows/                      CI + 판정 기준 가드
  ├── CLAUDE.md                               AI 에이전트용 프로젝트 지침
  └── docs/                                   이 문서
```

4-2. 왜 이렇게 나눴나 — "도메인형" 구조

처음에는 `controller/`, `service/`, `domain/`처럼 **역할별로** 나눴습니다. 그런데 여신이 들어오면서 바꿨습니다.

- **역할별(계층형)**: `service/` 폴더 안에 계좌·인증·대출 서비스가 전부 섞임
- **도메인형(지금)**: `loan/` 폴더만 열면 대출에 관한 **모든 것**이 다 있음

  12살 버전: 옷장을 정리할 때 "윗도리 서랍/아랫도리 서랍"으로 나누는 게 계층형이고, "여름 옷 칸/겨울 옷 칸"으로 나누는 게 도메인형입니다. 여름에 입을 걸 찾을 땐 후자가 훨씬 빠릅니다.

4-3. 의존 방향 — 화살표는 한쪽으로만

```
controller  →  service  →  repository  →  domain
  (얇게)      (규칙)      (DB 접근)     (자기 규칙을 스스로 지킴)
```

domain은 아무에게도 의존하지 않습니다. 계좌 객체는 스프링도, DB도 모르고 "잔액은 마이너스가 되면 안 된다" 같은 자기 규칙만 압니다.

왜 중요한가? 규칙이 여기저기 흩어지면 어디선가 규칙을 안 지키고 잔액을 바꿔버립니다. **잔액 변경은 오직 `Account.withdraw()` 같은 도메인 메서드로만** 하게 막았습니다.

---

5. 수신 파트 — 맡아주는 돈

5-1. 로그인은 어떻게 되나 (JWT)

문제

서버는 요청이 올 때마다 **"너 누구야?"**를 알아야 합니다. 그런데 매번 아이디/비밀번호를 보내면 위험하고, 서버가 로그인 상태를 전부 기억하면 서버가 여러 대일 때 골치가 아픕니다.

해결: JWT(출입증)

로그인에 성공하면 서버가 **출입증**을 하나 만들어 줍니다. 이 출입증에는 "이 사람은 aaa@bbb.com입니다"라는 내용과, 서버만 아는 비밀로 만든 **서명(도장)**이 찍혀 있습니다.

헤더.내용.서명 ← 점 두 개로 나뉜 긴 글자

다음부터는 요청할 때마다 이 출입증을 같이 보냅니다. 서버는 **도장이 진짜인지만 다시 계산해서** 확인합니다. 진짜면 "우리가 발급한 게 맞네" → 통과.

> **12살 버전**: 놀이공원 손목 밴드예요. 입구에서 한 번 표를 사면 밴드를 채워주고, 이후엔 밴드만 보여주면 됩니다. 직원이 "이 사람 표 샀나?" 명단을 뒤질 필요가 없어요. 밴드에 위조 방지 무늬가 있으니까요.

Refresh Token(연장 티켓)

출입증(Access Token)은 **1시간**만 유효합니다. 짧게 만든 이유는 혹시 도둑맞아도 피해가 적기 때문입니다. 대신 **7일짜리 연장 티켓**을 DB에 저장해 두고, 출입증이 만료되면 이걸로 새로 받습니다.

- 연장 티켓은 **DB에 있어야만 유효** → 서버가 언제든 강제로 없앨 수 있음(로그아웃)
- 재발급할 때마다 **옛 티켓은 버리고 새로 발급**(Rotation) → 훔친 티켓을 오래 못 씀

비밀번호 저장

비밀번호는 **절대 그대로 저장하지 않습니다.** BCrypt라는 방법으로 **되돌릴 수 없게 뭉갠 값**만 저장합니다.

> **12살 버전**: 과일을 갈아서 주스로 만들면 다시 사과로 되돌릴 수 없죠. 로그인할 때는 입력한 비밀번호를 똑같이 갈아서, 저장된 주스와 같은지만 비교합니다.

5-2. 계좌 만들기

계좌번호는 SecureRandom(예측하기 어려운 무작위)으로 만들고, **이미 있는 번호면 다시 뽑습니다**(최대 10번). 10번 다 실패하면 오류를 냅니다.

```
private String generateAccountNumber() {
    for (int i = 0; i < MAX_ACCOUNT_NUMBER_ATTEMPTS; i++) {
        String candidate = randomAccountNumber();
        if (!accountRepository.existsByAccountNumber(candidate)) return candidate;
    }
    throw new BusinessException("계좌번호 생성에 실패했습니다.");
}
```

5-3. 입금·출금·이체

모든 돈 이동은 이 세 가지를 **한 덩어리로** 처리합니다.

1. 잔액을 바꾼다
2. 원장에 기록을 남긴다
3. 둘 다 성공해야 확정한다 (**트랜잭션**)

트랜잭션이 뭔가요? "전부 되거나, 전부 안 되거나" 둘 중 하나만 되게 묶는 것입니다.

12살 버전: 친구에게 사탕을 주면서 내 사탕은 줄고 친구 사탕은 늘어야 하죠. 그런데 내 사탕만 줄고 친구 건 안 늘어나면? 사탕이 증발합니다. 트랜잭션은 **"둘 다 되든가, 둘 다 없던 일로 하든가"**를 보장합니다.

이체 한 번은 원장에 **두 줄**이 남습니다. 보내는 쪽 TRANSFER_OUT, 받는 쪽 TRANSFER_IN.

2026.10 보강 — 수신을 여신만큼 단단하게 (PR #19)

여신을 만든 뒤 계좌 코드를 다시 검토했더니, **수신 쪽이 여신보다 덜 단단한** 곳이 세 군데 있었습니다.

빈틈	왜 위험한가	고친 방법
이체에 멱등키가 없었다	상환은 "두 번 눌러도 한 번"인데, 버튼 두 번이 가장 위험한 **이체**가 무방비	상환에서 만든 멱등 실행기를 재사용 (8-9)
락을 건 **뒤에** 주인을 확인했다	남의 계좌 ID로 요청만 보내도 **그 계좌에 락이 걸린다.** 거절은 되지만 그동안 진짜 주인의 거래가 줄을 선다	주인 ID만 뽑는 **프로젝션으로 먼저 확인** → 통과한 요청만 락
잠긴 계좌도 입금·출금은 됐다	잠금 검사가 이체에만 있었다	입금·출금에도 ACCOUNT_LOCKED 검사

```
// 입금 — 권한 확인 → 락 순서
assertOwner(accountId, member, "본인 계좌만 입금할 수 있습니다.");   // 주인 ID 프로젝션: 락 없음,
↪  캐시 오염 없음
Account account = lockAccount(accountId);                          // 이 계좌 엔티티의 첫 조회 = 락 조회
if (account.isLocked()) {
    throw new BusinessException(ErrorCode.ACCOUNT_LOCKED, "잠금된 계좌에는 입금할 수 없습니다.");
}
```

왜 락 없이 주인을 확인해도 안전한가? 계좌의 주인은 **바뀌지 않는 값**이기 때문입니다. 잔액처럼 계속 바뀌는 값은 락을 걸고 읽어야 하지만, 주인은 언제 읽어도 같습니다. 게다가 ID만 읽는 프로젝션이라 1차 캐시에도 들어가지 않으므로, 뒤의 락 조회가 낡은 값을 받을 일도 없습니다(아래 5-4 문제 3과 같은 원리).

12살 버전: 사물함 주인이 아닌 사람이 "열어 주세요" 하면, 열쇠를 꽂아 보기 전에 **명단부터 확인**하고 돌려보내야죠. 열쇠를 꽂고 나서 확인하면, 그동안 진짜 주인은 사물함 앞에서 기다려야 합니다.

5-4. 동시성 — 이 프로젝트의 심장

여기가 가장 중요한 부분입니다. **세 가지 문제를 순서대로 하나씩** 해결했습니다.

문제 1: 잃어버린 갱신 (lost update)

두 사람이 동시에 같은 계좌에서 돈을 뺀다고 해봅시다.

```
잔액 10,000원

A: 잔액 읽음(10,000) ──┐
B: 잔액 읽음(10,000) ──┤  둘 다 10,000을 봤다!
A: 10,000-7,000=3,000 저장
B: 10,000-7,000=3,000 저장   ← A가 뺀 걸 못 봄

결과: 14,000원이 나갔는데 잔액은 3,000원 (7,000원 증발)
```

12살 버전: 두 사람이 동시에 저금통을 보고 "3천 원 있네!"라고 기억한 다음, 각자 3천 원씩 꺼내가는 겁니다. 저금통엔 3천 원밖에 없었는데 6천 원이 나왔어요.

해결: 비관적 락(Pessimistic Lock)

계좌를 읽을 때 **"내가 다 쓸 때까지 아무도 건드리지 마"**라고 자물쇠를 겁니다. SQL로는 SELECT ... FOR UPDATE입니다.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Account> findWithLockingById(Long id);   // SELECT ... FOR UPDATE
```

이러면 A가 끝날 때까지 B는 **줄을 서서 기다립니다.** B가 읽을 땐 이미 3,000원으로 바뀐 진짜 값을 봅니다.

실제로 측정한 결과 (잔액 20만 원, 50명이 동시에 1만 원씩 출금 시도 — 정상이면 20명만 성공):

방식	성공 로그	최종 잔액	오차
락 없음	10건	130,000원	**30,000원 증발**
비관적 락	**20건**	**0원**	**0원 (정확)**

락 없는 쪽은 데드락 에러(`SQLState 40001`)도 잔뜩 났습니다.

> **숫자에 대해**: 락 없는 쪽은 스레드 경쟁 타이밍에 따라 **실행할 때마다 숫자가 달라집니다**(README에 실린 측정에서는 15만 원이 남아 5만 원이 증발했습니다). 변하지 않는 것은 두 가지 — 락이 없으면 **돈이 사라진다**는 사실, 그리고 비관적 락은 **매번 20건 성공·0원 오차**라는 것입니다.

문제 2: 데드락 (교착 상태)

이체는 **계좌 두 개**를 잠가야 합니다. 그런데 이런 일이 생깁니다.

```
A→B 이체:  A 잠금 → B 잠그려 대기
B→A 이체:  B 잠금 → A 잠그려 대기

서로 상대가 놓기를 영원히 기다림 = 데드락
```

> **12살 버전**: 좁은 문에서 두 사람이 서로 "네가 먼저 비켜"라고 하며 안 움직이는 상황이에요. 영원히 안 끝납니다.

해결: 항상 같은 순서로 잠근다

계좌 ID가 작은 것부터 잠그기로 규칙을 정했습니다. 그러면 두 이체 모두 같은 순서로 잠그니까 서로 기다릴 일이 없습니다.

측정 결과: 반대 방향 동시 이체 **100회 시도, 데드락 0건**.

문제 3: 낡은 값 읽기 (stale read) — 가장 미묘한 버그

이건 진짜 찾기 어렵습니다.

JPA에는 **1차 캐시**가 있습니다. 같은 트랜잭션 안에서 같은 데이터를 두 번 읽으면, 두 번째는 DB에 안 가고 **처음 읽어둔 값을 그냥 줍니다.**

```
1) 계좌를 그냥 읽음 (잔액 10,000) → 캐시에 저장됨
2) 락을 걸고 다시 읽음 → 캐시가 10,000을 돌려줌 (DB엔 3,000인데!)
3) 낡은 값으로 계산 → 또 틀어짐
```

> **12살 버전**: 냉장고에 우유가 있는지 확인하고 방에 왔어요. 나중에 다시 "우유 있었나?" 생각할 때, 냉장고를 다시 열지 않고 **기억에 의존**하면? 그 사이 누가 다 마셨을 수도 있잖아요.

해결: 잔액이 걸린 계좌는 첫 조회를 반드시 락 조회로 한다

그런데 문제가 하나 더 있습니다. **받는 계좌의 ID를 알아야 순서를 정하는데, ID를 알려면 읽어야 합니다.** 읽으면 캐시에 들어가고요.

→ **ID만 뽑아오는 조회(프로젝션)**를 씁니다. 엔티티 전체를 안 읽으니 캐시에 들어가지 않습니다.

```java
@Query("select a.id from Account a where a.accountNumber = :accountNumber")
Optional<Long> findIdByAccountNumber(String accountNumber);   // ID만!
```

이 세 가지 해결책은 나중에 **대출 상환에서도 똑같이 재사용**됩니다. 그리고 2026년 10월에는 거꾸로, 상환에서 다듬은 멱등키를 수신으로 다시 가져오면서 "권한 확인 → 락" 순서도 바로잡았습니다(5-3 보강, 8-9).

---

6. 돈을 다루는 규칙 — BigDecimal 이야기

6-1. 왜 소수가 필요한가

처음엔 잔액을 Long(정수)으로 저장했습니다. 원 단위니까 소수가 필요 없었죠. 그런데 **이자 계산**이 들어오면서 문제가 생깁니다.

`1,000만 원 × 연 6% × 30일 / 365일 = 49,315.068...원`

소수가 나옵니다. 그리고 컴퓨터에서 소수를 다루는 흔한 타입(double)은 **오차가 있습니다.**

```
0.1 + 0.2 == 0.30000000000000004   // double의 세계
```

> **12살 버전**: 컴퓨터는 1/3을 0.3333...으로 딱 떨어지게 표현 못 해요. 돈 계산에서 이런 오차가 쌓이면 "1원이 어디 갔지?"가 됩니다.

6-2. 해결: BigDecimal + DECIMAL(19,4)

- 자바에서는 **`BigDecimal`** (오차 없는 소수 계산 타입)
- DB에서는 **`DECIMAL(19,4)`** (전체 19자리, 소수점 아래 4자리)

6-3. 전환 시점을 왜 그때로 잡았나

이자 계산이 들어오기 "직전"에 전환했습니다.

- 너무 일찍 하면: 필요도 없는데 코드가 복잡해짐
- 너무 늦게 하면: 이미 여러 모듈이 정수를 쓰고 있어서 **다 뜯어고쳐야 함**

6-4. 규칙을 한 곳에 모았다 — `MoneyPolicy`

반올림 규칙이 여기저기 흩어지면 **같은 계산인데 파일마다 결과가 달라집니다.** 그래서 규칙을 클래스 하나에 못 박았습니다.

```java
public final class MoneyPolicy {
    public static final int CALC_SCALE = 10;                // 중간 계산은 소수 10자리
    public static final RoundingMode CALC_ROUND = HALF_UP;   // 중간은 반올림
    public static final int WON_SCALE = 0;                  // 확정 금액은 원 단위
    public static final RoundingMode INTEREST_ROUND = DOWN;  // 이자는 원 미만 절사
    public static final BigDecimal DAYS_PER_YEAR = new BigDecimal("365");
    public static final BigDecimal MAX_LEGAL_RATE = new BigDecimal("0.20"); // 법정최고 20%
}
```

핵심 원칙: 중간 계산은 넉넉하게(10자리), **최종 확정 금액만 원 단위로 자른다.** 중간마다 자르면 오차가 쌓입니다.

---

7. 여신 파트 — 빌려주는 돈

7-1. 대출 상품 (loan_products)

"어떤 조건으로 빌려주는지"를 정의합니다.

컬럼	뜻	예
interest_rate	연이율	0.059 = 연 5.9%
overdue_extra_rate	연체 시 추가 이율	0.03 = +3%
min_amount / max_amount	빌릴 수 있는 범위	100만 ~ 5,000만
max_term_months	최대 기간	60개월
repayment_type	갚는 방식	원리금균등 / 원금균등 / 만기일시

7-2. 상환 방식 3가지 — 그림으로 이해하기

1,000만 원을 연 6%로 12개월 빌린다고 할 때, 갚는 방법이 세 가지입니다.

① 원리금균등 (EqualPayment) — 매달 같은 금액

매달 860,664원씩 12번
[redacted] ... (전부 같은 높이)

내는 돈은 같지만 **속 구성이 변합니다.** 처음엔 이자가 많고 원금이 적다가, 뒤로 갈수록 원금이 많아집니다.

> **12살 버전**: 매달 용돈에서 똑같이 만 원씩 갚는 거예요. 편하죠. 대부분의 주택담보대출이 이 방식입니다.

공식:

$$A = P \times \frac{r(1+r)^n}{(1+r)^n - 1}$$

(P=원금, r=월이율, n=개월수)

> **2026.10 발견 — 원리금균등이 깨지는 조합**: 불변식 하네스(13장)의 무작위 탐색 20,000건 중 **588건**에서 만기 전에 잔액이 **음수**가 됐습니다. 월 납입액은 원 단위 **반올림**, 매달 이자는 원 미만 **절사** — 이 몇십 전짜리 차이가 이자율로 **복리 누적**되면서, 소액(59원·연 20%·11개월)이나 고금리·초장기(100만 원·연 19.9%·387개월)에서 마지막 회차 원금이 음수(= 고객에게 돈을 돌려줌)가 된 것입니다. 지금은 **생성 중 잔액이 음수가 되면 거부**합니다(IllegalArgumentException). 실제 상품 범위(최소 100만 원·최대 60개월)에서는 거부가 한 건도 없고, 기존 계산 결과와 외부 계산기 대조값도 그대로입니다(13-5).

② 원금균등 (EqualPrincipal) — 원금만 같게

1회차 [redacted] (원금 833,333 + 이자 50,000)
2회차 [redacted] (원금 833,333 + 이자 45,833)
...
12회차 [redacted] (이자가 점점 줄어듦)

처음엔 많이 내고 갈수록 적게 냅니다. 총 이자는 원리금균등보다 적습니다.

③ 만기일시 (Bullet) — 이자만 내다가 마지막에 몰아서

1~11회차 [redacted] (이자 50,000원만)
12회차 [redacted] (원금 1,000만 + 이자)

> **12살 버전**: "일단 매달 조금씩만 내고, 나중에 한 번에 갚을게!" 방식이에요.

7-3. 끝수 조정 — 1원이 남는 문제

여기가 실무에서 진짜 중요한 부분입니다.

이자를 매번 **원 미만 절사**하면, 12번 쌓인 오차 때문에 마지막에 **잔액이 딱 0이 안 됩니다.**

해결: 마지막 회차가 끝수를 흡수한다.

- 12회차 원금 = **남은 잔액 전부**
- 그래서 12회차 상환액은 860,664원이 아니라 **860,662원** (2원 작음)

이걸 빼먹으면 **완납했는데 잔액이 ±수 원 남습니다.** 테스트로 못 박아 뒀습니다.

7-4. 검증을 4층으로 했다

이자 계산은 틀리면 안 되니까 **네 겹으로** 확인합니다. 1~3층은 처음부터 있었고, 4층은 2026년 10월에 더했습니다.

1층: 손으로 계산한 값과 비교

수식을 직접 계산해서 "1회차 이자는 50,000원, 원금은 810,664원"을 코드에 그대로 적고 비교합니다.

2층: 240가지 조합에서 불변식 검사

금액 4가지 × 금리 4가지(0% 포함) × 기간 5가지 × 방식 3가지 = **240 조합**에서 다음이 **항상** 성립하는지 확인합니다.

- 회차 원금을 다 더하면 = 대출 원금 (1원도 안 새야 함)
- 마지막 회차 후 잔액 = 정확히 0
- 잔액은 계속 줄어들기만 함 (늘어나면 버그)

  12살 버전: 특정 문제 하나만 풀어보는 게 아니라, **"어떤 숫자를 넣어도 이 규칙은 깨지지 않아야 한다"**를 240번 확인하는 겁니다.

지금 이 240조합은 13장의 공통 판정기로 바뀌어, 위 세 가지를 포함한 **불변식 11종**(MONEY-01, SCH-01~10)을 검사합니다.

3층: 바깥 계산기와 대조

인터넷의 공개 대출이자 계산기와 같은 조건으로 비교했습니다.

월 상환금 860,664원 — 정확히 일치.

회차별 이자는 몇 원 차이가 났는데, 원인을 분석해 보니 그 계산기는 **10원 미만 절사**, FinBank는 **원 미만 절사**라는 정책 차이였습니다. 수식은 같습니다.

4층: 기계가 고른 입력으로 판정 — 불변식 하네스 (2026.10)

1~3층은 전부 **사람이 고른 입력**입니다. 사람이 상상하지 못한 입력은 검사하지 못합니다. 그래서 입력을 **기계가 고르게** 했습니다.

- 불변식을 **25개**로 늘리고 ID를 붙여 한 파일에 모았다 (`LoanInvariant`)
- 고정 시드로 무작위 입력 **20,000건**을 만들어 같은 불변식으로 판정
- 결과: 240조합이 덮지 못한 영역(소액·초장기)에서 **진짜 버그 1건** — 위 7-2의 원리금균등 음수 잔액

정답을 몰라도 판정할 수 있다는 것이 핵심입니다. 자세한 구조는 13장에서 설명합니다.

7-5. 자동심사 — 룰을 클래스 하나씩

대출 신청이 들어오면 자동으로 심사합니다. 규칙은 2가지입니다.

룰	탈락 조건
`CreditLimitRule`	기존 대출 + 신청액 > 한도(5,000만)

룰	탈락 조건
NoActiveOverdueRule	이미 연체 중인 대출이 있다

심사와 입력 검증의 경계: 신청할 때 대출금을 받을 계좌를 직접 고르게 되어 있고, 그 계좌의 존재·소유권·잠금 여부는 **심사가 아니라 입력 검증**으로 막습니다(404/403/400). 남의 계좌를 지정한 것은 "심사에서 떨어질 일"이 아니라 애초에 성립하지 않는 요청이기 때문입니다. 심사 룰은 계좌가 유효하다는 전제 위에서 **신용 판단**(한도·연체)만 맡습니다.

> 초기에는 HasActiveAccountRule(활성 계좌 보유)이라는 룰이 있었습니다. 계좌를 신청 시 지정하도록 바꾸면서 이 룰은 **절대 발동할 수 없는 코드**가 되었고, 도달 불가능한 코드를 남기는 대신 삭제했습니다.

설계 포인트: 룰을 if문 덩어리로 쓰지 않고 **인터페이스 하나 + 구현 클래스 하나씩**으로 만들었습니다.

```
public interface LoanReviewer {
    Optional<String> reject(ReviewContext context);
    // 탈락이면 룰 코드를 반환, 통과면 비어있음
}
```

스프링이 List<LoanReviewer>로 **전부 자동 주입**하므로, **룰을 추가하려면 클래스만 하나 더 만들면 됩니다.** 서비스 코드는 손대지 않습니다.

> **12살 버전**: 심사 항목을 종이 한 장에 다 적어놓으면, 항목 추가할 때마다 그 종이를 고쳐야 해요. 대신 **항목마다 카드 한 장씩** 만들어 상자에 넣으면, 새 항목은 **카드만 추가**하면 됩니다.

7-6. 대출 실행 — 돈이 실제로 나가는 순간

승인된 신청을 **실행**하면 한 트랜잭션 안에서 이 일들이 전부 일어납니다.

1. 신청 상태를 EXECUTED로 바꾼다 (조건부 UPDATE — 뒤에서 설명)
2. 대출 계약(loan_accounts) 생성 — **금리·상환방식 스냅샷 저장**
3. 상환 스케줄 **전 회차를 미리 계산해서 저장** (12개월이면 12줄)
4. 신청할 때 지정한 입출금 계좌에 **원금 입금** (지정이 없는 V7 이전 신청은 회원의 첫 번째 활성 계좌로 폴백)
5. 대출 원장에 EXECUTION 기록

왜 스케줄을 미리 다 저장하나? "몇 회차에 얼마 내야 하는지"가 계약 시점에 확정되어야 하고, 나중에 계산식이 바뀌어도 **이미 맺은 계약의 스케줄은 변하면 안 되기 때문**입니다.

7-7. 상환 — 갚기

- **회차 상환**: 이번 달 낼 돈을 계좌에서 빼고, 회차를 PAID로, 대출 잔액을 원금만큼 줄임
- **순서 규칙**: 1회차가 남아있는데 2회차를 내려 하면 거부 (409)
- **전액 중도상환**: 남은 원금 + 경과이자 + 중도상환수수료를 한 번에

중도상환수수료 계산

수수료 = 잔여원금 × 수수료율 × (잔존일수 ÷ 총 대출일수)

남은 기간이 짧을수록 수수료가 줄어듭니다(체감 방식). 만기일에 갚으면 수수료 0원입니다.

> **12살 버전**: 은행은 "10달 동안 이자 받을 계획"으로 돈을 빌려줬는데, 5달 만에 갚으면 계획이 틀어져요. 그 손해를 조금 물어주는 게 수수료입니다. 다만 만기가 다 되어 갚으면 손해가 없으니 안 받습니다.

남은 회차는 지우지 않고 CANCELED로 남깁니다. "원래 어떤 계획이었는지"도 사실이기 때문입니다(원장 철학의 연장).

---

8. 멱등키 — “두 번 눌러도 한 번만”

8-1. 문제 상황

사용자가 **상환 버튼을 두 번 눌렀습니다.** 또는 인터넷이 끊겨서 앱이 자동으로 재시도했습니다. 서버에는 **똑같은 요청이 두 번** 도착합니다.

돈이 두 번 빠지면 안 됩니다.

> **12살 버전**: 자판기에서 버튼을 두 번 눌렀는데 음료수가 두 개 나오고 돈도 두 번 빠지면 억울하잖아요. “이미 처리한 요청이면 그냥 아까 그 음료수를 다시 보여줘”가 멱등입니다.

8-2. 해결: 요청마다 고유 번호표

클라이언트(앱/웹)가 **요청 하나마다 UUID(고유 번호)**를 만들어 헤더에 담아 보냅니다.

```
Idempotency-Key: 620f6171-0ece-4307-a953-77d732d3ca3d
```

서버는 이 번호표를 보고 판단합니다.

- **처음 보는 번호** → 처리하고, 결과를 저장해 둠
- **이미 처리한 번호** → 저장해 둔 결과를 **그대로 다시 돌려줌** (돈은 안 움직임)
- **처리 중인 번호** → 409 (잠시 후 다시)
- **같은 번호인데 내용이 다름** → 422 (번호표 재사용은 거부)

8-3. 화면에서 키를 어떻게 다뤄야 하나

여기서 **처음에 실수**를 했습니다. API를 호출할 때마다 새 UUID를 만들고 있었습니다.

```
// 잘못된 방식: 호출마다 새 키 → 멱등이 전혀 안 걸림
fetch(url, { headers: { 'Idempotency-Key': crypto.randomUUID() } })
```

올바른 방식은 “사용자 행동 1회 = 키 1개”입니다. 버튼을 누른 순간 키를 만들고, 네트워크 실패로 재시도할 때는 **같은 키를 그대로 다시** 보냅니다. 지금은 이 규칙을 공통 모듈 `api/idempotency.js` 하나에 모아 이체·대출 실행·상환이 같이 씁니다(부록 A-7).

8-4. 트랜잭션 경계 — 가장 어려운 부분

멱등키를 제대로 만들려면 **“어디까지가 한 덩어리인가”**를 정교하게 나눠야 합니다.

단계	전파 속성	왜
키 선점(INSERT)	`REQUIRES_NEW`(새 트랜잭션)	본 처리와 같이 묶으면 롤백 시 **키도 사라져** 동시 재시도를 못 막는다
결과 확정	`MANDATORY`(같은 트랜잭션)	“돈이 움직인 사실”과 “저장된 응답”이 **같은 커밋에 묶여야** 한다
실패 시 해제	`REQUIRES_NEW`	롤백되는 트랜잭션에 얹으면 **삭제도 함께 취소**된다

> **12살 버전**: 도서관에서 책을 빌릴 때 “예약 도장”을 먼저 찍어야 남이 못 가져가요. 그런데 그 도장을 **대출 절차와 한 묶음**으로 만들면, 대출이 취소될 때 도장도 지워져서 그 틈에 남이 채갑니다. 그래서 **도장은 따로** 찍습니다.

8-5. 경쟁의 심판은 DB에게 맡긴다

“먼저 조회해서 없으면 저장”은 **동시에 오면 둘 다 통과합니다.**

```
A: 키 있나? → 없음
B: 키 있나? → 없음      (A가 아직 저장 전)
A: 저장 → 성공
B: 저장 → 성공          ← 둘 다 통과! 실패
```

그래서 **UNIQUE 제약**을 걸어두고 **그냥 INSERT를 시도**합니다. DB가 **반드시 하나만** 통과시킵니다.

> **12살 버전**: 의자 뺏기 게임에서 "빈자리 있나 두리번거리고 앉기"는 두 명이 같이 앉을 수 있어요. 그냥 **앉아버리면** 의자는 하나니까 한 명만 앉습니다. 심판을 사람이 아니라 **의자에게 맡기는** 거죠.

8-6. 실제 측정 결과

100개 스레드가 같은 회차를 동시에 상환 시도 (1회차 총액 860,205원):

시나리오	성공 응답	409 충돌	기타 예외	원장	실제 출금	잔액 감소
서로 다른 100개 요청	**1**	99	0	1건	1회	860,205원
같은 키 100번 재시도	70	30	0	**1건**	**1회**	860,205원

두 번째 줄이 멱등키의 존재 이유입니다. 100개 중 70개가 **성공 응답을 받았지만 돈은 한 번만 움직였습니다.** 69개는 저장된 응답의 재생이고, 30개는 선점 경쟁 중이라 409를 받았습니다.

이체도 같은 시험을 통과합니다 (2026.10, `TransferIdempotencyTest`): 100만 원 계좌에서 7만 원 이체를 **같은 키로 100개 동시에** 보내면, 응답은 "성공" 아니면 "처리 중(409)"뿐이고(기타 예외 0), 출금 원장 **1건**·입금 원장 **1건**, 보낸 쪽 잔액은 **정확히 7만 원만** 줄어듭니다. 성공과 409의 비율은 실행마다 달라지지만, 돈이 한 번만 움직인다는 결과는 매번 같습니다.

8-7. 이중 처리 방어 — 층마다 다른 걸 막는다

사고	1차 방어	최종 방어
같은 신청 이중 실행	조건부 UPDATE (0행이면 409)	`application_id` UNIQUE
같은 회차 이중 상환	조건부 UPDATE	`idempotency_key` UNIQUE
버튼 두 번 / 재시도	멱등키 선점	위 두 제약
같은 이체 두 번 / 재시도 (2026.10)	멱등키 선점	`idempotency_keys` UNIQUE + 응답 확정을 이체와 같은 트랜잭션에서

조건부 UPDATE가 왜 핵심인가

MySQL 기본 격리수준(REPEATABLE READ)에서 **평범한 SELECT는 트랜잭션 시작 시점의 사진(스냅샷)을 봅니다.** 락을 기다렸다 통과한 두 번째 요청이 **"이미 상환된 회차"를 아직 안 낸 것으로 볼 수 있다**는 뜻입니다.

반면 **UPDATE는 항상 최신 값을 기준으로** 조건을 평가합니다.

```
UPDATE repayment_schedules
   SET status = 'PAID'
 WHERE id = ? AND status IN ('PENDING','OVERDUE')   -- 조건을 UPDATE에 넣는다
```

바뀐 행 수가 **0이면 "이미 누가 했다"** → 409로 거부.

> **핵심**: **상태 확인과 상태 변경을 DB의 한 문장으로 합치는 것**이 경쟁을 없애는 방법입니다. 확인과 변경 사이에 틈이 있으면 그 틈으로 사고가 들어옵니다.

8-8. 대출 상환의 락 순서

상환은 **계좌와 대출 두 자원**을 동시에 잠급니다. 전역 규칙은 **`Account` → `LoanAccount` 순서**입니다.

그런데 연결계좌 ID는 대출을 읽어야 압니다 → 수신에서 배운 대로 **ID만 프로젝션으로** 먼저 읽습니다. 엔티티를 먼저 읽으면 1차 캐시 때문에 락 조회가 낡은 값을 돌려주니까요.

8-9. 이체로 확장 — 같은 장치, 같은 원칙 (PR #19)

처음에는 멱등키가 **여신(실행·상환)에만** 있었습니다. 그런데 버튼 두 번이 가장 위험한 곳은 사실 **이체**입니다. 그래서 상환에서 만든 멱등 실행기를 이체에 그대로 씌웠습니다. 구조도 상환과 똑같이 **껍데기 + 본체**입니다. 껍데기에는 트랜잭션이 없고, 본체가 트랜잭션 안에서 이체와 응답 확정을 함께 합니다.

```
// AccountTransferService — 껍데기: 키 선점 · 해제는 각자 별도 트랜잭션 (그래서 여기엔 트랜잭션이 없다)
public TransferResponse transfer(TransferRequest request, String idempotencyKey) {
    Long memberId = currentMember.currentId();
    String amount = request.getAmount() == null ? null
            : request.getAmount().stripTrailingZeros().toPlainString();   // 70000 == 70000.00
    String hash = RequestHash.of("account-transfer", memberId,
            request.getFromAccountId(), request.getToAccountNumber(), amount);

    return idempotentExecutor.run(idempotencyKey, memberId, hash, TransferResponse.class,
            () -> accountService.transfer(request, idempotencyKey));      // 본체 (@Transactional)
}
```

설계	이유
요청 해시에 출금 계좌·받는 계좌·금액을 모두 넣는다	같은 키로 **금액만 바꿔** 보내면 422. 조용히 옛 응답을 돌려주면 사용자는 바꾼 금액이 이체됐다고 착각한다
금액을 stripTrailingZeros로 정규화	70000과 70000.00은 같은 요청 — 표기 차이로 422가 나면 안 된다
응답 확정(complete)을 이체와 **같은 트랜잭션**에서	커밋 직후 서버가 죽어도, 재시도가 이체를 한 번 더 하지 않는다
실패하면 키 해제	잔액 부족으로 실패한 키로, 입금 후 다시 시도할 수 있어야 한다
응답을 TransferResponse record로	저장했다가 그대로 재생하려면 역직렬화할 수 있는 형태여야 한다. 기존 message 필드는 호환을 위해 유지

TransferIdempotencyTest 5개가 이 표를 한 줄씩 고정합니다 — 순차 재시도 재생, 금액 변경 422, 표기 차이 동일 취급, 실패 시 키 해제, 100개 동시 재시도.

> **배포할 때 주의**: 이체 API에 Idempotency-Key 헤더가 **필수**가 되면서 API 계약이 바뀌었습니다. 헤더를 보내지 않는 옛 화면은 400을 받으므로, 프론트엔드와 백엔드를 **함께** 배포해야 합니다.

---

9. 검색과 인덱스 — 느린 걸 빠르게

9-1. 문제: 조건이 7개인 검색

대출 목록을 상태·상환방식·상품·실행일 구간·원금 구간으로 자유롭게 조합해서 찾고 싶습니다. 조건을 넣을 수도, 안 넣을 수도 있습니다.

조합의 수가 100가지를 넘습니다. 이걸 `if`문으로 쓰면 코드가 지옥이 됩니다.

9-2. 해결: QueryDSL

조건 하나 = 메서드 하나로 만들고, 값이 없으면 `null`을 반환합니다. QueryDSL의 `where()`는 **null을 만나면 그 조건을 그냥 건너뜁니다.**

```
private BooleanExpression statusEq(LoanAccountStatus status) {
    return status == null ? null : loanAccount.status.eq(status);
}
```

> **12살 버전**: 검색 조건을 카드로 만들어서, 빈 카드는 그냥 버리는 거예요. 조합이 몇 가지든 카드는 조건 개수만큼만 있으면 됩니다.

9-3. 인덱스 — 책의 목차

인덱스가 없으면 DB는 표를 **처음부터 끝까지 다 훑습니다**(풀스캔).

> **12살 버전**: 두꺼운 책에서 "고양이"라는 단어를 찾을 때, 1쪽부터 넘기면 오래 걸리죠. 뒤에 **색인(찾아보기)**이 있으면 "고양이 → 45쪽"으로 바로 갑니다. 인덱스가 바로 그 색인입니다.

9-4. 실제로 재봤다 (10만 건)

조건: 한 회원이 대출 10만 건 보유 / "최근 30일 안에 실행된 정상 대출"(전체의 약 2.7%)

쿼리	인덱스	실행 방식	조사한 행 수	평균 시간
COUNT	없음	풀스캔(ALL)	96,880	26.62ms
COUNT	**복합 인덱스**	범위 스캔(range)	**2,664**	**1.04ms**

조사 행 수 36배 감소, 시간 26배 단축.

9-5. 인덱스 컬럼 순서가 중요하다

인덱스는 `(member_id, status, executed_at)` 순서로 만들었습니다. 순서에 이유가 있습니다.

> **핵심 규칙**: **등치 조건(=)을 앞에, 범위 조건(>, <, BETWEEN)을 뒤에.**
>
> **왜?** 범위 조건 뒤에 오는 컬럼은 **탐색 범위를 더 좁히지 못하기 때문**입니다.
>
> **12살 버전**: 반 친구를 찾을 때 "3학년 → 2반 → 키 순서"로 정렬돼 있으면 빨리 찾죠. 그런데 "키 순서 → 3학년 → 2반"으로 정렬하면? 키로 한 번 범위를 잡은 다음엔 학년·반이 뒤죽박죽이라 소용이 없습니다.

9-6. 정직하게 기록한 한계

정렬이 붙은 목록 쿼리(`ORDER BY id DESC LIMIT 10`)는 **인덱스가 있으나 없으나 PK를 탔습니다.**

옵티마이저가 "PK 역순으로 훑다가 10건 채우면 끝"이 더 빠르다고 판단한 겁니다. 지금은 조건 일치율이 2.7%라 빠르지만, **데이터 분포에 의존하는 계획**입니다.

→ **커버링 인덱스나 커서 페이징이 필요한 지점**으로 문서에 남겨뒀습니다. 이런 한계를 숨기지 않고 적는 게 중요합니다.

---

10. 배치 — 밤에 혼자 도는 프로그램

10-1. 배치가 뭔가요?

사람이 시키지 않아도 **정해진 시간에 자동으로 도는 프로그램**입니다. API가 "손님 한 명의 요청"을 그 자리에서 처리한다면, 배치는 **"조건에 맞는 전부"를 한꺼번에** 처리합니다.

> **12살 버전**: 선생님이 수업 중에는 질문한 학생에게 한 명씩 답해 주지만(API), 밤에는 반 전체 숙제를 한꺼번에 채점해요(배치). 손님이 기다리지 않는 시간에 몰아서 하는 게 효율적입니다.

FinBank는 **매일 새벽 2시**에 두 개가 순서대로 돕니다.

순서	잡(Job)	읽는 것	하는 일	남기는 것
①	`overdueDetectionJob` 연체 감지	납기가 지났는데 아직 PENDING인 회차	회차 → OVERDUE, 대출 → OVERDUE	연체 이력(`overdue_histories`)
②	`overdueInterestAccrualJob` 연체이자 가산	해소되지 않았고 오늘 아직 이자를 안 붙인 연체 이력	밀린 일수만큼 연체이자를 계산해 누적	대출 원장 `OVERDUE_INTEREST` 거래

순서가 중요합니다. 오늘 새로 연체된 회차는 감지가 이력을 만들어 줘야 가산 대상이 됩니다. 순서를 뒤집으면 오늘 연체된 건의 이자가 **하루 늦게** 붙습니다(기산점이 납기일이라 금액은 다음 날 따라잡지만, 그날 밤 장부는 실제보다 적습니다).

10-2. Spring Batch의 뼈대 — Job · Step · 청크

```
OverdueBatchScheduler   (@Scheduled 매일 02:00, JobParameters: baseDate=오늘)
  │
  ├──▶ overdueDetectionJob
  │      └── overdueDetectionStep          청크 100건마다 커밋
  │           Reader     OverdueScheduleReader    납기 지난 PENDING 회차 (커서)
  │           Processor  회차 · 대출 OVERDUE + 연체 이력 open
  │           Writer     연체 이력 saveAll
  │
  └──▶ overdueInterestAccrualJob
         └── overdueInterestAccrualStep     청크 100건마다 커밋
              Reader     OverdueHistoryReader     미해소 & 오늘 미가산 이력 (커서)
              Processor  경과일수 → 연체이자 계산 → history.accrue
              Writer     대출 원장(OVERDUE_INTEREST) saveAll
```

부품	하는 일	12살 버전
Job	배치 작업 하나 (이름 + 단계 목록)	"오늘 밤 할 일"
Step	작업 안의 한 단계	"수학 숙제 채점"
Reader	처리할 것을 하나씩 읽어 온다. 더 없으면 `null` → 스텝 종료	숙제 더미에서 한 장 꺼내기
Processor	하나를 처리한다. `null`을 돌려주면 그 건은 저장하지 않는다	한 장 채점하기
Writer	청크 단위로 모아서 한 번에 저장한다	채점한 100장을 한 번에 서류함에 넣기
청크(chunk)	한 트랜잭션에 묶는 건수 — 이만큼마다 커밋	100장마다 서류함 잠그기
JobParameters	실행할 때 넘기는 값 (기준일, 청크 크기)	채점지에 적는 "기준 날짜"
JobRepository (`BATCH_*` 표)	언제·어떤 파라미터로 돌았는지, 성공했는지 기록	채점 기록부

왜 청크로 끊나? 10만 건을 한 트랜잭션으로 묶으면 마지막 1건이 실패할 때 앞의 99,999건까지 되돌아갑니다. 청크로 끊으면 **실패 지점까지는 커밋되어 있고**, 다시 돌리면 남은 것만 처리됩니다(10-7의 멱등 덕분에 안심하고 다시 돌릴 수 있습니다).

스텝은 이렇게 조립합니다.

```
@Bean
@JobScope   // 잡을 실행할 때마다 빈을 새로 만든다 → 그때의 파라미터(chunkSize)를 읽을 수 있다
public Step overdueDetectionStep(JobRepository jobRepository,
                                 PlatformTransactionManager transactionManager,
                                 OverdueScheduleReader overdueScheduleReader,
                                 @Value("#{jobParameters['chunkSize'] ?: 100}") int chunkSize) {
    return new StepBuilder(STEP_NAME, jobRepository)
            .<RepaymentSchedule, OverdueHistory>chunk(chunkSize, transactionManager)
            .reader(overdueScheduleReader)
            .processor(overdueProcessor())
            .writer(overdueWriter())
            .build();
}
```

@JobScope / @StepScope가 왜 필요한가? 스프링 빈은 기본적으로 앱이 켜질 때 **한 번** 만들어집니다. 그러면 청크 크기나 기준일 같은 파라미터가 기동 시점 값으로 굳어 버립니다. 스코프를 걸면 **실행할 때마다 새로** 만들어져 그날의 파라미터를 읽습니다. 특히 Reader는 "어디까지 읽었나(lastId)"라는 상태를 들고 있으므로 매 실행마다 새것이어야 합니다.

10-3. 스케줄러 — 언제, 무엇을, 어떤 순서로

```
@Scheduled(cron = "0 0 2 * * *")          // 초 분 시 일 월 요일 → 매일 02:00:00
public void runDailyOverdueSettlement() {
    LocalDate baseDate = LocalDate.now();
    JobParameters params = new JobParametersBuilder()
            .addString("baseDate", baseDate.toString())   // 파라미터는 기준일 하나
            .toJobParameters();

    run(overdueDetectionJob, params, "연체 감지");          // ① 먼저
    run(overdueInterestAccrualJob, params, "연체이자 가산"); // ② 그다음
}

private void run(Job job, JobParameters params, String label) {
    try {
        jobLauncher.run(job, params);
    } catch (Exception e) {
        log.error("[{}] 배치 실행 실패", label, e);   // 배치 실패가 앱을 죽이면 안 된다
    }
}
```

설계 결정이 세 개 들어 있습니다.

1. **"오늘"은 스케줄러 한 곳에서만 정하고, 배치 로직(Reader·Processor)은 파라미터로 받는다.** 그래서 테스트에서 날짜를 마음대로 줄 수 있고, 특정 날짜 기준으로 **다시 돌릴 수 있습니다**(10-10).
2. **파라미터를 기준일 하나로 둔 것은 의도다.** Spring Batch는 "같은 파라미터 = 같은 JobInstance"로 보고 **이미 성공한 인스턴스의 재실행을 거부**합니다. 같은 날 두 번 트리거돼도 두 번째는 프레임워크가 막습니다.
3. **실패해도 앱은 산다.** 예외를 잡아 기록만 하고 끝냅니다. 밀린 일은 다음 실행이 처리합니다(멱등이라 안전).

설정 파일에도 두 줄이 있습니다.

```
spring:
  batch:
    job:
      enabled: false              # 앱이 켜질 때마다 배치가 자동 실행되면 안 된다 — 시점은 스케줄러가
↪   정한다
    jdbc:
      initialize-schema: always   # BATCH_* 메타 표는 프레임워크 소유 — 프레임워크가 만들게 둔다
```

BATCH_* 표는 왜 Flyway로 관리하지 않나? 프레임워크 버전이 올라가면 그 표의 모양도 같이 바뀝니다. 우리 마이그레이션에 복사해 두면 업그레이드 때마다 두 곳을 맞춰야 합니다. 그래서 **도메인 표는 Flyway가, 프레임워크 표는 프레임워크가** 관리하도록 나눴습니다.

10-4. 연체 감지 — 상태를 바꾸고 이력을 연다

```
// Processor — 상태 전이의 판단은 도메인 메서드가 한다
schedule -> {
    schedule.markOverdue();                    // 회차 PENDING → OVERDUE
    schedule.getLoanAccount().markOverdue();   // 대출 ACTIVE → OVERDUE
    return OverdueHistory.open(schedule, schedule.getDueDate());   // 연체 시작일 = 납기일
}

// Writer — 새로 생긴 이력만 저장한다
histories -> {
    overdueHistoryRepository.saveAll(histories.getItems());
    log.info("[연체 감지] {}건 전이", histories.size());
}
```

- **연체의 기준**: 납기일 **다음 날**부터입니다(due_date < 기준일). 납기일 당일은 아직 연체가 아닙니다.
- **연체 원리금**: 이력을 열 때 그 회차의 납입액(원금 + 이자)을 적어 둡니다. 연체이자는 이 금액을 기준으로 붙습니다.
- **Writer가 회차·대출을 저장하지 않는 이유**: 둘 다 이미 영속 상태라, 청크 트랜잭션이 커밋될 때 **변경 감지**로 같이 반영됩니다. 새로 만든 이력만 saveAll 하면 됩니다.
- **인덱스를 먼저 깔아 뒀다**: V6의 idx_repayment_schedules_status_due (status, due_date)가 이 배치의 주 경로입니다. 배치가 도는 시점에 상환 회차는 대출 건수 × 회차 수만큼 쌓여 있기 때문입니다(대출 10만 건 × 12회차 = 120만 행).

10-5. 가장 조심해야 할 함정: 페이징 밀림

이 배치들은 **읽은 행의 상태를 바꿔서, 자기 조회 조건에서 스스로 빠져나갑니다.** 여기서 흔한 방식(offset 페이징)을 쓰면 **조용히 데이터가 빠집니다.**

```
1페이지(0~99번) 처리 → 이 100건이 조건에서 빠짐
2페이지(offset 100) 조회
   → 원래 100~199번이던 행들이 0~99번으로 당겨져 있음
   → 그 100건을 통째로 건너뛴다!
```

10만 건 중 절반만 처리하고도 배치는 "정상 종료"로 끝납니다.

> **12살 버전**: 사탕 100개를 한 줄로 세워놓고 "앞에서 10개씩 가져가기"를 하는데, 가져간 사탕은 줄에서 빠져요. 그럼 "11번째부터"라고 세면 이미 줄이 당겨져서 **엉뚱한 사탕부터** 가져갑니다. 중간에 건너뛴 사탕이 생기죠.

해결: 커서 방식 — "마지막으로 읽은 id **다음부터**" 읽습니다.

```
public class OverdueScheduleReader implements ItemReader<RepaymentSchedule> {

    private final Deque<RepaymentSchedule> buffer = new ArrayDeque<>();
    private long lastId = 0L;                      // 위치(offset)가 아니라 마지막 id를 기억한다

    @Override
    public RepaymentSchedule read() {
        if (buffer.isEmpty()) fill();
        return buffer.poll();                      // 비어 있으면 null → 스텝 종료
    }

    private void fill() {
        List<RepaymentSchedule> next = repository.findOverdueCandidates(
                baseDate, lastId, PageRequest.of(0, chunkSize));   // 페이지 번호는 늘 0
        if (next.isEmpty()) return;
        buffer.addAll(next);
        lastId = next.get(next.size() - 1).getId();
    }
}

// RepaymentScheduleRepository.findOverdueCandidates
@Query("""
        select s from RepaymentSchedule s
```

```
          join fetch s.loanAccount
         where s.status = com.finbank.backend.loan.domain.ScheduleStatus.PENDING
           and s.dueDate < :today
           and s.id > :lastId
         order by s.id asc
        """)
```

이러면 앞이 빠져도 상관없고, offset이 커질수록 느려지는 문제도 없어집니다(10만 번째 페이지도 첫 페이지와 같은 비용). `join fetch`는 Processor가 대출 상태까지 바꾸므로 대출을 같이 가져와 N+1을 막습니다. **청크 경계를 넘는 250건 테스트**로 못 박아 뒀습니다.

> **두 번째 함정 — 조회 단위를 청크 크기와 같게 맞춘 이유**: Reader가 한 번에 청크보다 많이 읽어 두면, 앞 청크의 트랜잭션에서 읽은 엔티티가 다음 청크에서는 **준영속**(JPA가 더는 추적하지 않는 상태)이 됩니다. 그러면 Processor가 바꾼 상태가 변경 감지로 반영되지 않습니다. 에러도 없이 **조용히 누락**됩니다. 그래서 Reader는 청크 크기만큼만 읽습니다.

왜 Spring Batch가 주는 `JpaPagingItemReader`를 안 썼나? 그게 바로 offset 페이징이기 때문입니다. 읽은 행이 조건에서 빠지는 배치에는 맞지 않습니다. 연체이자 가산의 `OverdueHistoryReader`도 같은 이유로 커서 방식입니다 — 그쪽도 읽은 행의 `lastAccruedAt`을 바꿔 스스로 조건에서 빠지기 때문입니다.

10-6. 연체이자 가산 — 밀린 날만큼, 겹치지 않게

`연체이자 = 연체 원리금 × min(약정금리 + 연체가산율, 법정최고 20%) × 경과일수 / 365`

읽는 조건이 곧 멱등 장치입니다.

```
// OverdueHistoryRepository.findAccrualTargets
@Query("""
        select h from OverdueHistory h
          join fetch h.loanAccount
         where h.resolvedAt is null
           and (h.lastAccruedAt is null or h.lastAccruedAt < :baseDate)
           and h.id > :lastId
         order by h.id asc
        """)
```

- `resolvedAt is null` — 해소되지 않은 연체만
- `lastAccruedAt < :baseDate` — 오늘 아직 이자를 안 붙인 것만
- `join fetch h.loanAccount` — Processor가 대출의 금리·가산율을 읽으므로 같이 가져온다

```
// Processor
history -> {
    LocalDate from = (history.getLastAccruedAt() != null)
            ? history.getLastAccruedAt()          // 마지막으로 붙인 날부터
            : history.getOverdueStart();          // 처음이면 납기일부터
    long days = ChronoUnit.DAYS.between(from, baseDate);
    if (days <= 0) return null;                   // 붙일 구간이 없다

    LoanAccount loan = history.getLoanAccount();
    BigDecimal penalty = overdueInterestCalculator.calculate(
            history.getOverdueAmount(), loan.getInterestRate(), loan.getOverdueExtraRate(), days);

    history.accrue(penalty, baseDate);            // 0원이어도 기산점은 전진시킨다
    return penalty.signum() == 0 ? null : LoanTransaction.overdueInterest(loan, penalty);
}
```

코드	왜 이렇게
`from` = 마지막 가산일, 없으면 납기일	구간이 정확히 이어져 **겹치지도 비지도 않는다**
`return null`	Spring Batch 규칙: Processor가 `null`을 주면 Writer로 넘어가지 않는다 → **0원짜리 원장은 남기지 않는다**
0원이어도 accrue	기산점을 전진시켜야 다음 실행에서 같은 구간을 다시 계산하지 않는다

코드	왜 이렇게
`OVERDUE_INTEREST` 거래	원장은 append-only — 연체이자도 거래로 남는다. 단 **대출 원금 잔액은 변하지 않는다**
계산은 `OverdueInterestCalculator`	순수 자바 계산기. 배치는 "언제·무엇을"만 정하고 "얼마"는 계산기가 정한다 → 같은 계산기를 13장의 하네스가 검사한다(OVD-01~04)

기산점이 이어지는 모습:

```
7/1  연체 시작 (납기일)
7/11 배치 → 10일치 가산, lastAccruedAt = 7/11
7/16 배치 → 7/11~7/16의 5일치만 가산   ← 겹치지도 비지도 않음
```

배치가 사흘 밀렸다면 다음 실행에서 3일치가 한 번에 붙습니다.

10-7. 멱등을 세 겹으로

배치가 실수로 두 번 돌아도 사고가 나면 안 됩니다.

층	장치	막는 것
프레임워크	Spring Batch JobInstance	같은 파라미터(기준일)의 재실행 거부
감지 배치	`status = PENDING` 조회 조건	이미 전이된 회차는 조회조차 안 됨
가산 배치	`lastAccruedAt < 기준일` 조건	오늘 이미 가산한 이력은 제외

원리는 하나입니다: 처리 완료 표시를 데이터에 남기고, 그 표시를 읽기 조건에 넣는다.

2·3번째 줄은 **프레임워크 방어가 뚫려도** 버팁니다. 테스트처럼 파라미터를 바꿔 같은 날 다시 돌려도, 이미 전이된 회차와 오늘 가산한 이력은 조회 조건에서 빠집니다. 도메인 메서드도 한 번 더 막습니다 — 회차의 `markOverdue()`는 `PENDING`일 때만, 대출의 `markOverdue()`는 `ACTIVE`일 때만 상태를 바꿉니다.

> **12살 버전**: 숙제를 다 하면 공책에 도장을 찍어요. 다음에 "도장 없는 것만" 찾으면, 실수로 두 번 확인해도 이미 한 건 다시 안 합니다.

10-8. 상환과 맞물리는 지점 — 연체 해소

배치가 연체를 **여는** 쪽이라면, 상환은 연체를 **닫는** 쪽입니다.

```
// LoanRepaymentProcessor — 회차를 갚을 때 (같은 트랜잭션 안)
withdrawFromAccount(account, total);
loan.repayPrincipal(principal);
resolveOverdueHistory(target);    // 이 회차의 진행 중 연체 이력 → resolvedAt = 오늘 → 가산 대상에서
↪ 빠짐
resolveOverdueIfCleared(loan);    // 연체 회차가 하나도 안 남으면 대출 OVERDUE → ACTIVE
```

이 두 줄은 처음부터 있던 게 아닙니다. 배치를 만들고 나서야 **"연체 회차를 갚아도 이자가 계속 붙는"** 빈틈이 드러났습니다(실패 기록 ⑥). 지금은 "연체 회차를 상환하면 이력이 해소되고 가산이 멈춘다" 테스트가 이 연결을 지킵니다.

10-9. 청크 사이즈 — 재보고 정했다

청크(chunk)란 "한 트랜잭션에 몇 건을 묶을 것인가"입니다. 크게 잡으면 커밋(저장 확정) 횟수가 줄어 빨라지지만, 실패했을 때 되돌아가는 범위가 커집니다.

5만 건 연체 감지 실측:

청크 크기	처리시간	커밋 횟수	건당
100	37,969ms	500	759µs
500	12,418ms	100	248µs

청크 크기	처리시간	커밋 횟수	건당
1000	**9,927ms**	**50**	**198μs**

100 → 1000으로 3.8배 빨라졌습니다. 그런데 자세히 보면:

- 100 → 500: **3.1배** 빨라짐
- 500 → 1000: **1.25배**밖에 안 빨라짐

수확이 급격히 체감합니다. 커밋 비용이 지배적인 구간을 벗어나면, 그다음은 쿼리와 메모리 비용이라 청크를 키워도 얻는 게 적습니다.

그래서 정답이 아니라 트레이드오프입니다. 기본값 100을 유지하되 잡 파라미터(chunkSize)로 조정할 수 있게 열어뒀습니다.

10-10. 운영 관점 — 놓쳤을 때, 실패했을 때, 다시 돌릴 때

상황	어떻게 되나	왜
서버가 꺼져 하루치 실행을 놓쳤다	**다음 날 실행이 저절로 따라잡는다**	감지는 "기준일 이전 납기 **전부**"를 보고, 가산은 lastAccruedAt부터 계산하므로 밀린 일수가 한 번에 붙는다
같은 날 두 번 트리거됐다	두 번째는 실행되지 않고 에러 로그만 남는다	같은 파라미터(기준일)로 성공한 JobInstance는 재실행 거부
중간에 실패했다	실패한 청크만 되돌아가고 그 앞은 커밋된 상태	청크 단위 커밋. 다음 실행은 읽기 조건이 처리된 건을 빼 주므로 **남은 것만** 처리한다
특정 날짜 기준으로 다시 판정해야 한다	그 날짜를 baseDate로 주고 실행	배치 로직이 "오늘"을 직접 읽지 않으므로 가능 (테스트로 확인)
얼마나 처리했나 보고 싶다	로그와 BATCH_* 표	청크마다 [연체 감지] N건 전이 / [연체이자 가산] N건 기록 로그, 실행 이력·상태·건수는 메타 표에

> **실패한 실행은 같은 파라미터로 다시 돌릴 수 있나?** 네. Spring Batch가 거부하는 것은 **성공한** 인스턴스의 재실행뿐이고, 실패한 인스턴스는 재시작할 수 있습니다. FinBank의 Reader는 "어디까지 읽었나"를 따로 저장하지 않지만, 처음부터 다시 읽어도 처리된 행은 조회 조건에서 이미 빠져 있으므로 결과가 같습니다.

지금 없는 것도 적어 둡니다. 운영자가 화면이나 API로 배치를 직접 실행하는 기능은 없습니다. 실행 주체는 스케줄러 하나뿐이고, 날짜를 지정한 재실행은 테스트 코드로만 확인했습니다. 실제 운영이라면 관리자용 실행 기능이나 외부 스케줄러(10-11)가 필요합니다.

10-11. 스케줄러의 한계를 정직하게 적었다

@Scheduled는 **인스턴스마다 독립적으로 돕니다.** 서버를 2대로 늘리면 같은 배치가 동시에 두 번 트리거됩니다.

- 지금 안전한 이유: **단일 인스턴스 배포**이고, 위의 멱등 3겹이 있어서 중복 실행이 사고로 이어지지 않음
- 다중화한다면: 분산 락(ShedLock)이나 외부 스케줄러(쿠버네티스 CronJob)로 실행 주체를 하나로 만들어야 함
- **지금 도입하지 않는 이유**: 다중 인스턴스를 증명할 환경이 없어서

10-12. 테스트 12개가 지키는 것

연체 감지 OverdueDetectionJobTest (5개)

테스트	지키는 것
납기 지난 미납 회차만 OVERDUE로 전이되고 연체이력이 생긴다	판정 기준 (그 밖의 회차는 그대로)
같은 기준일로 두 번 돌려도 연체이력이 늘지 않는다	멱등

테스트	지키는 것
청크 경계를 넘어도 한 건도 빠뜨리지 않는다 (250건)	커서 Reader
연체 대상이 없으면 아무 일도 일어나지 않는다	빈 입력에서 정상 종료
기준일을 과거로 주면 그날 기준으로 판정한다	놓친 배치 재실행

연체이자 가산 `OverdueInterestAccrualJobTest` (7개)

테스트	지키는 것
연체 10일 후 실행 → 계산기와 같은 금액이 누적되고 원장에 남는다	금액 정확성
같은 기준일로 두 번 돌려도 이자가 두 번 붙지 않는다	멱등
이어서 실행: 10일 + 5일 = 15일치	구간 연속성
배치가 밀려도 다음 실행에서 밀린 일수만큼 한 번에 가산된다	따라잡기
해소된 연체는 더 이상 가산되지 않는다	해소
연체 회차를 상환하면 이력이 해소되고 가산이 멈춘다	상환 ↔ 배치 연결
연체 당일 실행 → 경과일수 0이므로 가산도 원장도 없다	0일 경계

여기에 성능 측정 `BatchChunkSizePerformanceTest`(`perf` 태그)가 5만 건에서 청크 100/500/1000을 비교하면서, **어떤 청크 크기에서도 5만 건이 전부 처리되는지**도 함께 확인합니다. 테스트에서는 잡 파라미터에 UUID를 더해 "같은 파라미터 재실행 거부"를 피합니다(실패 기록 ⑤).

---

11. 예외 처리 — 오류를 어떻게 알려주나

11-1. 상태 코드로 구분한다

코드	뜻	예
400	요청이 잘못됨	잔액 부족, 금액이 0 이하
403	권한 없음	남의 계좌를 보려 함
404	없음	없는 계좌
409	**충돌**	이미 실행됨, 이미 상환됨, 처리 중
422	**처리 불가**	멱등키 재사용, 금액이 상품 범위 밖

409와 422를 따로 만든 이유: 400 하나로 뭉뚱그리면 화면에서 "다시 시도해도 되는 오류"와 "다시 시도하면 안 되는 오류"를 구분할 수 없습니다.

11-2. ErrorCode로 한 곳에 모았다

```
public enum ErrorCode {
    // 400 — 요청 자체가 규칙 위반 (계좌: 2026.10 문자열 예외에서 코드로 전환)
    INVALID_AMOUNT(BAD_REQUEST, "금액은 0보다 커야 합니다."),
    INSUFFICIENT_BALANCE(BAD_REQUEST, "잔액이 부족합니다."),
    SAME_ACCOUNT_TRANSFER(BAD_REQUEST, "같은 계좌로는 이체할 수 없습니다."),
    ACCOUNT_LOCKED(BAD_REQUEST, "잠금된 계좌가 있습니다."),

    // 409 — 지금 상태와 충돌
    INVALID_STATE(CONFLICT, "현재 상태에서는 처리할 수 없습니다."),
    ALREADY_EXECUTED(CONFLICT, "이미 실행된 대출 신청입니다."),
    ALREADY_REPAID(CONFLICT, "이미 상환된 회차입니다."),
    OUT_OF_ORDER_INSTALLMENT(CONFLICT, "회차는 순서대로만 상환할 수 있습니다."),
    IDEMPOTENT_REQUEST_IN_PROGRESS(CONFLICT, "동일한 요청이 처리 중입니다."),

    // 422 — 형식은 맞지만 처리할 수 없음
    AMOUNT_OUT_OF_RANGE(UNPROCESSABLE_ENTITY, "허용 범위를 벗어난 금액입니다."),
    TERM_OUT_OF_RANGE(UNPROCESSABLE_ENTITY, "허용 범위를 벗어난 대출 기간입니다."),
    IDEMPOTENCY_KEY_MISMATCH(UNPROCESSABLE_ENTITY, "같은 멱등키로 다른 내용의 요청을 보낼 수
    ↪  없습니다.");
}
```

서비스는 **의미에 맞는 예외만 던지고**, 상태 코드 매핑은 `GlobalExceptionHandler` 한 곳에 모았습니다. 계좌 예외는 2026년 10월에 코드 기반으로 바꿨는데, **상태 코드(400)는 그대로 두고 코드 이름만 세분화**했습니다. 화면이 받던 응답 형태를 깨지 않으면서, 잔액 부족과 잠금 계좌를 구분할 수 있게 됐습니다.

11-3. 점진 이관 (expand-contract)

기존에는 `BusinessException(String)`으로 문자열만 던지고 있었습니다. 이걸 전부 한 번에 바꾸면 위험하니까:

1. 기존 생성자를 **그대로 두고**
2. `ErrorCode`를 받는 생성자를 **추가**

→ 기존 코드는 하나도 안 고치고 새 코드부터 새 방식을 씁니다.

> **12살 버전**: 좁은 다리를 새 다리로 바꿀 때, 헌 다리를 부수고 새로 짓는 게 아니라 **옆에 새 다리를 놓고 사람들을 천천히 옮기는** 방식입니다.

11-4. 알고 있는 빈틈 — 엔진의 거부가 API까지 오면

상환 스케줄 엔진은 계약 밖의 입력을 `IllegalArgumentException`으로 거부합니다(13장). 그런데 이 예외는 `GlobalExceptionHandler`에 따로 매핑되어 있지 않아, **API까지 올라오면 500**이 됩니다.

지금은 문제가 되지 않습니다. 신청 단계 검증(상품 한도·기간 → 422)이 먼저 걸러 주므로 **실제 상품 범위에서는 엔진까지 가는 거부가 0건**입니다. 다만 장기 상품을 추가해 엔진이 거부할 수 있는 입력이 API로 들어오게 되면, 이 예외를 **422로 매핑**해야 합니다. 다음 할 일로 기록해 뒀습니다.

---

12. 테스트 — 508개가 무엇을 지키나

12-1. 철학: 개수보다 "위험한 곳"

모든 코드를 다 테스트하지 않았습니다. **틀렸을 때 돈이 사라지는 곳**에 집중했습니다. 그리고 2026년 10월부터는 "사람이 고른 입력"의 한계를 넘기 위해 **불변식 하네스**(13장)를 더했습니다.

508개는 `./gradlew test`가 실제로 실행하는 수입니다 — `@Test` 메서드 112개 + 파라미터 테스트 실행 396회(240조합 + 계산기 143조합 + 케이스 파일 13건). 성능 측정 2개(`perf` 태그)는 평소 제외합니다.

묶음	개수
수신 (동시성·규칙·멱등)	22
예외	3
여신 계산 엔진	273
불변식 하네스	161
여신 통합 (실제 DB)	37
배치	12
합계	**508**

이 중 계산 엔진 패키지(`domain/calculator`)와 불변식 하네스를 합친 **428개**가 DB 없이 수십 초면 도는 **빠른 테스트**(`fast-test.sh`)입니다. AI 에이전트가 실행할 수 있는 유일한 명령이기도 합니다(14장).

> **지금 브랜치에서는 4개가 일부러 실패합니다.** 입력 계약(INP-01~04)을 판정 기준에 **먼저** 넣어 두고 구현을 아직 하지 않은 상태이기 때문입니다(14-7). 구현 라운드가 끝나면 전부 통과해야 하고, 그때 PR을 엽니다.

12-2. 테스트 파일별 역할 (전체)

수신 — 동시성과 규칙

파일	개수	무엇을 지키나
`ConcurrencyBenchmarkTest`	1	**락 없음 vs 비관적 락** 비교표 출력 (증발 금액을 숫자로)
`WithdrawConcurrencyTest`	1	10만 원 계좌에 7만 원 출금 2번 동시 → **1번만 성공**
`AccountConcurrencyTest`	1	동시 이체 → 1건만 성공
`AccountServiceDeadlockTest`	1	반대 방향 동시 이체 100회 → **데드락 0건**
`TransactionLogTest`	3	입금·출금·이체 시 원장이 정확히 남는가 (이체는 2줄)
`TransferRuleTest`	10	잔액부족·같은계좌·남의계좌·잠금계좌 거부 + **잠금 계좌 입출금 차단**, 남의 계좌 입출금은 **락을 걸기 전에 403**
`TransferIdempotencyTest`	5	같은 키 순차 재시도 → 재생, 금액만 바꾸면 422, `70000` = `70000.00`, 실패 시 키 해제, **같은 키 100개 동시 → 이체 1회**

예외

파일	개수	무엇을 지키나
`GlobalExceptionHandlerTest`	3	예외 → 상태코드 매핑, 레거시 호환

여신 — 계산 엔진 (DB 없이 순수 자바)

파일	개수	무엇을 지키나
ScheduleGeneratorTest	14	3가지 상환방식의 회차별 원금·이자·잔액 **확정값** + 끝수 조정 + **회귀 2건**(59원·20%·11개월, 100만 원·19.9%·387개월 → 거부)
ScheduleGeneratorPropertyTest	**240 조합**	공통 판정기로 불변식 11종(MONEY-01, SCH-01~10)이 **항상** 성립
AccruedInterestCalculatorTest	5	경과이자 계산 + 원 미만 절사
OverdueAndPrepaymentCalculatorTest	8	연체이자(법정최고 20% 캡) + 중도상환수수료(체감)
LoanApplicationTest	6	신청 상태 전이 규칙 (APPLIED→APPROVED→EXECUTED)

불변식 하네스 (DB 없이, 2026.10~)

파일	개수	무엇을 지키나
CalculatorInvariantsTest	**143 조합**	일할 계산기 3종(경과이자·연체이자·중도상환수수료)을 일수 전 구간에서 — 0 경계·단조성·상한
ScheduleCaseFileTest	13	케이스 파일 2개(사람 10건 + AI 3건)를 같은 판정기로 — ok면 불변식, reject면 거부
CaseFileFormatTest	4	케이스 파일 형식 가드레일 — 열 9개, ID 형식·중복, ok 케이스는 입력 계약 안
RandomExplorationTest	1	고정 시드 42 × 20,000건 — 계약 밖은 반드시 거부, 계약 안은 불변식 준수

여신 — 통합 (실제 DB)

파일	개수	무엇을 지키나
LoanApplicationServiceTest	11	자동심사 2룰(한도·연체) + 상품 한도·기간 검증(422) + **입금 계좌 지정 검증**(없음 404·남의 계좌 403·잠금 400) + 취소
LoanExecutionServiceTest	7	실행 정합성, **멱등 재생**, 이중 실행 409, 키 재사용 422, 실패 시 키 해제
LoanRepaymentServiceTest	9	회차 상환, 순서 규칙, 멱등, 완납, 전액 중도상환
LoanRepaymentConcurrencyTest	2	**100스레드 동시 상환 → 1건만**, 같은 키 100회 → 돈은 1번만
LoanSearchTest	8	조건 조합, 경계 포함, 타인 대출 미노출, 페이징

배치

파일	개수	무엇을 지키나
OverdueDetectionJobTest	5	감지 정확성, 멱등, **청크 경계 250건 누락 없음**, 기준일 파라미터
OverdueInterestAccrualJobTest	7	계산기와 금액 일치, 멱등, **구간 연속성**, 밀린 배치 따라잡기, **상환 시 가산 중단**

성능 측정 (perf 태그 — 평소엔 제외)

파일	무엇을 재나
LoanSearchPerformanceTest	10만 건 인덱스 전/후 EXPLAIN + 응답시간
BatchChunkSizePerformanceTest	5만 건 청크 100/500/1000 처리시간

12-3. 왜 진짜 MySQL로 테스트하나

메모리 DB(H2)를 쓰면 테스트가 훨씬 빠릅니다. 그런데 **안 씁니다.**

이 프로젝트가 증명하려는 것들이 대부분 DB 엔진의 특성이기 때문입니다.

- 비관적 락(`SELECT ... FOR UPDATE`)의 동작
- 조건부 UPDATE가 최신 값을 보는 것
- REPEATABLE READ 스냅샷

H2로 바꾸면 **검증 자체가 무의미해집니다.**

12-4. 성능 테스트를 분리한 이유

10만 건을 적재하는 측정은 1분 넘게 걸립니다. 매 커밋마다 돌리면 **개발 피드백이 느려집니다.**

```
test    { useJUnitPlatform { excludeTags 'perf' } }   // 평소: perf 제외
perfTest { useJUnitPlatform { includeTags 'perf' } }   // 필요할 때만
```

> **CI 시간 = 개발 피드백 속도**입니다.

12-5. 테스트 DB 분리

테스트는 `DatabaseCleaner`로 **모든 표를 비웁니다.** 개발용 DB를 그대로 쓰면 `./gradlew test` 한 번에 개발하며 만든 데이터가 전부 사라집니다.

→ 테스트는 **`finbank_test`** 스키마, 개발 서버는 **`finbank`** 스키마로 갈랐습니다.

> **왜 중요한가?** 테스트가 "돌리기 무서운 것"이 되면 **결국 안 돌리게 됩니다.**

---

13. 불변식 하네스 — 정답을 몰라도 틀린 걸 잡는 법

> 2026년 10월에 추가한 부분입니다(PR #20). 14장의 AI 에이전트 하네스가 "AI가 만든 코드와 케이스를 무엇으로 판정하나"에 답하려면, 이 장치가 먼저 있어야 했습니다.

13-1. 문제: 정답을 누가 정하나

보통의 테스트는 **입력 하나에 정답 하나**를 적습니다. "1,000만 원·연 6%·12개월이면 월 상환금은 860,664원" 같은 식입니다(7-4의 1층). 확실하지만 한계가 두 가지 있습니다.

1. **사람이 상상한 입력만 검사합니다.** 사람이 떠올리지 못한 조합에 버그가 숨어 있으면 영원히 못 찾습니다.
2. **입력이 수천 개가 되면 정답을 적을 수 없습니다.** 무작위로 만든 2만 개, AI가 만든 케이스의 정답을 누가 손으로 계산하나요?

이걸 **테스트 오라클 문제**라고 합니다. 오라클(oracle)은 "이 결과가 맞다/틀리다"를 판정하는 기준입니다.

> **12살 버전**: 수학 문제 1,000개를 받았는데 답지가 없어요. 그래도 틀린 답은 찾을 수 있습니다. "두 양수를 더한 답은 둘 중 큰 수보다 작을 수 없다", "뺄셈 답에 뺀 수를 더하면 원래 수가 나와야 한다" 같은 **항상 맞는 규칙**으로 검산하면 되니까요. 그 규칙이 **불변식**입니다.

13-2. 불변식 — 입력이 무엇이든 항상 참인 규칙

대출 계산에서는 이런 것들입니다.

ID	규칙	깨지면 생기는 일
INV-SCH-02	회차 원금을 다 더하면 대출 원금	1원이 새거나 생긴다
INV-SCH-03	마지막 회차 잔액 == 0	완납했는데 잔액이 남는다
INV-SCH-04	회차 원금·이자 ≥ 0	고객에게 돈을 돌려주는 회차가 생긴다
INV-SCH-06	회차 잔액 == 직전 잔액 − 회차 원금	장부가 중간에 끊긴다
INV-SCH-07	i회차 납기일 == 실행일 + i개월	말일 실행에서 납기가 밀린다
INV-OVD-03	연체 적용 금리 ≤ 연 20%	법정최고금리 위반
INV-PRE-02	일찍 갚을수록 수수료는 줄지 않는다	체감 방식이 거꾸로 된다

정답(860,664원)을 몰라도 **이 규칙들이 깨졌는지는 언제나 판정할 수 있습니다.** 그래서 사람이 고른 입력이든, 무작위 입력이든, AI가 만든 입력이든 **같은 기준**으로 판정합니다.

카탈로그 — 25개를 ID와 함께 한 파일에

```java
public enum LoanInvariant {
    MONEY_01("MONEY-01", "확정 금액은 0 이상의 원 단위 정수다 (MoneyPolicy: 원 미만 절사)"),

    INP_01("INP-01", "원금은 0보다 큰 원 단위 정수"),
    INP_02("INP-02", "연이율은 0 이상, 법정 최고금리 연 20% 이하 (정확히 20%는 허용)"),
    INP_03("INP-03", "대출 기간은 1개월 이상 600개월(50년) 이하"),
    INP_04("INP-04", "실행일은 필수이고, 만기일이 날짜 표현 범위 안에 있어야 한다"),

    SCH_01("INV-SCH-01", "회차 수 == 대출 기간, 회차 번호는 1부터 연속"),
    SCH_02("INV-SCH-02", "Σ(회차 원금) == 대출 원금 — 반올림으로 1원도 새거나 늘지 않는다"),
    SCH_03("INV-SCH-03", "마지막 회차 잔액 == 0 — 완납 시 끝수가 남지 않는다"),
    // ... SCH-04 ~ SCH-10, PRE-01 ~ 03, OVD-01 ~ 04, ACC-01 ~ 03
}
```

묶음	개수	무엇을 지키나
MONEY	1	공통 금액 정책
INP	4	엔진이 받아들일 입력의 경계 (13-6)

묶음	개수	무엇을 지키나
SCH	10	상환 스케줄 (3가지 방식 공통)
PRE	3	중도상환수수료
OVD	4	연체이자
ACC	3	경과이자

왜 한 파일에, ID까지 붙여서?

- **단일 출처** — 문서·실패 리포트·AI에게 주는 피드백이 모두 같은 ID(`INV-SCH-03`)로 말합니다.
- **지키기 쉽다** — 14장에서 AI가 판정 기준을 못 고치게 막는데, 기준이 여기저기 흩어져 있으면 막아야 할 경로도 흩어집니다.

  불변식의 한계: 정책 자체를 **일관되게** 잘못 이해했다면 불변식은 통과할 수 있습니다. 그래서 손으로 계산한 확정값(1층)과 외부 계산기 대조(3층)는 그대로 둡니다. 불변식은 그 위에서 **"사람이 상상하지 못한 입력"**을 맡는 층입니다.

13-3. 판정기 — JUnit 없이, 위반을 전부 모은다

```
// 스케줄 판정기 — 결과를 받아 위반 목록을 돌려준다 (비어 있으면 통과)
List<Violation> found = ScheduleInvariants.check(type, principal, rate, term, executedAt, lines);
```

위반 한 줄은 **"어떤 규칙이, 어디서, 어떻게"** 깨졌는지 담습니다. 형식은 `규칙 ID ¦ 위치와 실제 값 (규칙 설명)`입니다.

```
MONEY-01 ¦ 10회차: 잔액 -1 (확정 금액은 0 이상의 원 단위 정수다 (MoneyPolicy: 원 미만 절사))
```

설계	이유
JUnit에 의존하지 않는 평범한 자바 클래스	테스트·스크립트·AI 에이전트 라운드가 **같은 판정기**를 재사용한다
첫 위반에서 멈추지 않고 **전부** 모은다	앞의 몇 줄이 원인, 나머지는 연쇄 효과인 경우가 많다 — 전체를 봐야 원인이 보인다
위반 메시지에 실제 값	사람이 읽고 바로 재현할 수 있다. 이 메시지가 그대로 **구현 AI에게 돌아가는 피드백**이 된다

판정기는 두 개입니다. `ScheduleInvariants`는 상환 스케줄(MONEY-01, SCH-01~10)을, `CalculatorInvariants`는 일할 계산기 3종(PRE·OVD·ACC)을 봅니다. 계산기 쪽은 일수를 0일부터 전 구간에 걸쳐 늘려 가며 단조성·상한·0 경계를 검사합니다.

13-4. 입력은 네 군데서 온다

입력 출처	무엇	누가 관리	건수
매트릭스 (코드)	금액 4 × 금리 4 × 기간 5 × 방식 3	사람	240
`schedule-cases.csv`	사람이 고른 경계값(H-) + 실제로 버그를 잡은 입력(R-)	사람	10
`llm-cases.csv`	검증 AI가 코드를 읽고 노린 입력(L-)	검증 에이전트 (끝에 추가만)	3
무작위 탐색	고정 시드 42로 만든 입력	기계	20,000

케이스 파일은 한 줄에 입력·기대·근거를 다 적습니다.

```
id,type,principal,rate,term,executedAt,expect,source,note
H-003,BULLET,10000000,0.068,12,2026-01-31,ok,manual,말일 실행 — 2월 납기 월말 보정
H-007,EQUAL_PAYMENT,999999999999,0.199,12,2026-01-15,ok,manual,초대형 원금 — 중간 계산 정밀도
R-001,EQUAL_PAYMENT,59,0.2,11,2026-08-31,reject,regression,반올림 누적으로 10회차 잔액 -1원 …
L-001,EQUAL_PRINCIPAL,1000.5,0.05,12,2026-01-15,reject,llm,원 미만 원금 — validate가 정수 여부를 …
```

- **expect** — ok면 스케줄이 만들어지고 불변식을 전부 지켜야 하고, reject면 엔진이 IllegalArgumentException으로 거부해야 합니다. **정답 숫자는 어디에도 없습니다.**
- **매트릭스는 코드, 개별 케이스는 CSV** — 240조합을 CSV 240줄로 풀면 관리가 안 됩니다. 반대로 "왜 이 입력인가"가 중요한 개별 케이스는 한 줄에 근거(note)까지 적는 CSV가 리뷰하기 좋습니다.
- **CSV를 둘로 나눈 이유** — 검증 AI에게 **"AI용 파일 끝에 추가만"** 권한을 주기 위해서입니다. 사람 파일은 판정 기준이라 AI가 고칠 수 없습니다(14장).
- **형식 가드레일** CaseFileFormatTest — 열 9개, ID 형식(H-/R- + 숫자 3자리, L- + 숫자), ID 중복 금지, ok 케이스는 입력 계약 안. 일부러 잘못 만든 행 12종을 넣어 가드가 실제로 잡는지도 확인합니다(가드의 가드).

13-5. 무작위 탐색이 찾은 진짜 버그

무작위 탐색은 사람이 고르지 않을 입력을 섞습니다.

```
private static final long SEED = 42L;            // 고정 시드 → 누가 돌려도 같은 2만 건
private static final int CASES = 20_000;
private static final long[] PRINCIPALS = {1, 2, 7, 11, 59, 99, 100, 101, 999, 1000, 12345,
       999999, 1000001, 3333333, 99999999, 999999999999L};  // 절반은 경계값, 절반은 1원~10억 원 무작위
// 금리: 0, 0.0000001, 0.199, 0.2, 0.5, 1, 2.5 … 또는 0~30% 무작위 / 기간: 1~12 또는 1~480개월
// 실행일: 1/15, 1/31, 2028-02-29(윤일), 8/31, 12/31
```

처음 돌렸을 때 **20,000건 중 588건**에서 MONEY-01(확정 금액은 0 이상)이 깨졌습니다. 전부 **원리금균등**에서, 만기 전에 잔액이 음수가 되는 조합이었습니다.

원인을 따라가 보면

1. 매달 낼 돈 A는 공식으로 구해 **원 단위로 반올림**합니다(최대 0.5원 많게).
2. 매달 이자는 **원 미만을 절사**합니다(최대 1원 적게).
3. 그 달 원금 = A − 이자. 그래서 원금이 공식보다 **조금씩 더** 갚아집니다.
4. 그 차이는 잔액에 남아 다음 달 이자 계산에 들어가므로 **이자율만큼 불어납니다**(복리).
5. 금액이 아주 작거나(59원·20%·11개월), 금리가 높고 기간이 아주 길면(100만 원·19.9%·387개월) 끝나기 전에 잔액이 0을 지나 **음수**가 됩니다 → 마지막 회차 원금이 음수 = **고객에게 돈을 돌려주는 회차**.

사람이 고른 240조합은 왜 못 찾았나? 그 조합들은 "상품이 허용하는 현실적인 입력" 근처에 모여 있었고, 버그는 그 바깥(소액·초장기)에 있었습니다.

어떻게 고쳤나 — 고치지 않고 거부했다

선택지	문제
판정 기준을 느슨하게 (음수 허용)	고객에게 돈을 돌려주는 스케줄을 "정상"이라 부르게 된다
회차마다 납입액을 다시 계산	고칠 범위가 크고, 기존 결과·외부 계산기 대조값이 바뀐다
조용히 틀리지 말고 거부한다 (채택)	실제 상품 범위(100만 원 이상·60개월 이하)는 영향 0, 기존 값 그대로

```
// EqualPaymentGenerator — 생성 중 잔액이 음수가 되면 거부
for (int i = 1; i < termMonths; i++) {
    BigDecimal interest = MoneyPolicy.toWon(balance.multiply(r));   // 이자: 원 미만 절사
    BigDecimal principalPortion = payment.subtract(interest);       // 원금 = A − 이자
    balance = balance.subtract(principalPortion);
    if (balance.signum() < 0) {
        throw new IllegalArgumentException("원리금균등 스케줄을 만들 수 없는 조건입니다 (...): "
                + "반올림 누적으로 " + i + "회차에 잔액이 음수가 됩니다.");
    }
    ...
}
```

결과: 588건이 **전부 거부로 바뀌고 위반은 0건.** 두 대표 입력은 회귀 케이스 R-001, R-002와 ScheduleGeneratorTest의 회귀 테스트 2개로 고정했습니다. 장기 상품(주택담보 등)을 추가할 때는 잔여 회차 재산정 방식으로 재설계가 필요하다는 것도 코드 주석에 남겼습니다.

> **재검증 (2026-11-04)**: 머지된 코드로 무작위 10만 건을 따로 돌려도 위반 0건(거부는 전부 원리금균등·잔액 음수 사유), 실제 상품 범위 전수 9,000조합에서는 거부 0건이었습니다.

13-6. 입력 계약 — 엔진이 받아들일 입력의 경계

14장의 검증 AI가 첫 라운드에서 **엔진이 받아들이면 안 되는 입력이 통과하는** 곳을 찾았습니다. 사람이 그걸 보고 **입력 계약**을 정했습니다.

ID	규칙	발견 계기
INP-01	원금은 0보다 큰 원 단위 정수	L-001: 1000.5원이 통과 → 마지막 회차 원금이 87.5원
INP-02	연이율 0 이상, 연 20% 이하 (정확히 20%는 허용)	L-002: 연 20.00001%가 통과
INP-03	기간 1개월 이상 600개월 이하	기간 상한이 없어 약 21억 개월이면 OutOfMemoryError
INP-04	실행일 필수, 만기일이 날짜 표현 범위 안	L-003: 서기 약 10억 년 실행일 → 계약 밖 예외(DateTimeException)

계약 밖의 입력은 `IllegalArgumentException`으로 거부해야 합니다. 판정 기준은 이렇게 생겼습니다.

```
public final class InputContract {
    /** 법정 최고금리 연 20% — 정확히 20%는 허용 */
    public static final BigDecimal MAX_ANNUAL_RATE = new BigDecimal("0.20");
    /** 대출 기간 상한 600개월(50년) — 상품 정책이 아니라 엔진이 감당할 수 있는 범위 */
    public static final int MAX_TERM_MONTHS = 600;

    /** 위반한 계약 목록. 비어 있으면 계약 안의 입력이다. */
    public static List<LoanInvariant> violations(BigDecimal principal, BigDecimal annualRate,
                                                 int termMonths, LocalDate executedAt) { ... }
}
```

판정 기준은 구현의 상수를 참조하지 않습니다. 구현 쪽에도 `MoneyPolicy.MAX_LEGAL_RATE`(20%)가 있지만 일부러 가져다 쓰지 않고 따로 적었습니다. 기준이 구현의 상수에 기대면, **구현이 상수를 바꾸는 순간 기준도 같이 느슨해지기** 때문입니다.

> **12살 버전**: 채점 기준표에 "정답은 학생 답안지 3번 칸을 보라"고 적어 두면, 학생이 3번 칸을 고치는 순간 채점 기준도 바뀌어 버립니다. 기준표에는 정답을 **직접** 적어야 합니다.

600개월은 왜? 상품 정책(지금 최대 60개월)이 아니라 **엔진이 감당할 범위**로 잡았습니다. 상품이 바뀔 때마다 엔진을 고치지 않게 하기 위해서입니다.

무작위 탐색의 판정도 이 계약에 맞춰 바뀌었습니다.

- 계약 **밖**의 입력 → 반드시 거부되어야 한다
- 계약 **안**의 입력 → 불변식을 모두 지키거나, "잔액 음수" 사유로만 거부될 수 있다

13-7. 이 하네스가 다루지 않는 것

- **정책이 일관되게 틀린 경우** — 1층(손계산 확정값)과 3층(외부 계산기 대조)이 보완합니다.
- **DB가 필요한 영역** — 계좌·동시성·멱등은 12장의 통합 테스트(실제 MySQL)가 맡습니다. 하네스는 순수 자바인 계산 엔진만 대상으로 합니다. 빠르고 결정적이어야 AI에게 즉시 피드백을 줄 수 있기 때문입니다(14장).
- **고정 시드의 양면** — 시드가 고정이라 누가 돌려도 같은 결과(재현성)지만, 늘 같은 2만 건만 봅니다. 새 영역은 더 큰 탐색을 따로 돌려 확인합니다(위의 재검증).

---

14. AI 에이전트 하네스 — 만드는 AI와 검증하는 AI를 나누다

> 2026년 11월 현재 진행 중인 부분입니다(`feature/agent-harness` 브랜치). 무엇이 끝났고 무엇이 남았는지는 14-9에 정리했습니다.

14-1. 왜 이걸 했나 — "AI가 만든 결과를 어떻게 믿나"

금융 IT 채용 공고에서 **AI 활용 역량**을 보는 곳이 늘었습니다. 그런데 금융권이 실제로 걱정하는 것은 "AI를 쓸 줄 아느냐"보다 **"AI가 만든 결과를 어떻게 믿느냐"**입니다. 돈 계산이 틀린 코드가 AI 덕분에 더 빨리 들어오면 오히려 사고입니다.

그래서 AI를 서비스에 붙이는 대신, **AI가 FinBank의 코드를 고치게 하되 틀린 것은 들어오지 못하게 하는 환경**을 만들었습니다. 이런 일을 **하네스 엔지니어링**이라고 부릅니다.

> **하네스(harness)**는 말에 씌우는 마구입니다. 힘센 말(AI)이 엉뚱한 데로 달리지 않게, 힘은 그대로 쓰되 방향을 잡아 주는 장치입니다. 모델을 바꾸는 게 아니라, 모델이 일하는 **환경**(역할·권한·피드백·판정 기준)을 설계합니다.

하지 않은 것과 이유

후보	하지 않은 이유
RAG 챗봇	흔해서 차별화가 약하다. 살펴본 공고 중 RAG를 요구한 곳이 없었다
FDS(이상거래탐지)	가짜 데이터라 탐지 성능 수치에 의미를 붙이기 어렵다
서비스에 LLM 연동	범위가 크고, "이게 없으면 뭐가 안 되는데?"에 답하기 어렵다 (2-3의 원칙)

> **12살 버전**: 시험을 생각해 보세요. **응시자**(구현 AI)는 답안을 쓰고, **채점자**(검증 AI)는 까다로운 문제를 더 내고 채점합니다. **채점 기준표**(불변식)는 선생님(사람)만 고칠 수 있고, **교실 문**(hook)이 응시자가 채점 기준표에 손대지 못하게 막습니다. 응시자가 아무리 똑똑해도 기준표를 고쳐서 100점을 받을 수는 없습니다.

14-2. 원칙 세 가지

1. **만드는 쪽과 검증하는 쪽을 나눈다 (maker-checker)** — 자기 코드를 자기가 검증하면 같은 맹점을 공유합니다. 은행 업무에서 "입력한 사람과 승인하는 사람을 나누는" 것과 같은 원칙입니다.
2. **판정은 결정적인 코드가 한다** — 흐름 제어와 판정은 스크립트·JUnit·hook·CI가 하고, AI는 구현·케이스 만들기·판단만 합니다. 기준은 AI의 "통과했어요"라는 말이 아니라 **테스트 결과**입니다.
3. **판정 기준은 사람만 고친다 (오라클 보호)** — AI가 테스트를 느슨하게 고쳐서 통과시키는 것이 가장 흔한 실패입니다. 이걸 **부탁이 아니라 구조로** 막습니다.

14-3. 누가 무엇을 할 수 있나

주체	읽기	수정	실행
구현 에이전트 `loan-implementer`	비밀 파일 외 전체	`finbank-backend/src/main/**`만	`fast-test.sh`만
검증 에이전트 `loan-verifier`	비밀 파일 외 전체	`llm-cases.csv`(끝에 추가만), `harness-reports/**`(`task.md` 제외)	`fast-test.sh`만
사람	비밀 파일 외 전체	전체 (판정 기준은 `FINBANK_ALLOW_ORACLE_EDIT=1` + PR 라벨)	Claude Code 기본 권한 절차

판정 기준(오라클) — 어떤 에이전트도 고칠 수 없는 파일들입니다.

경로	왜 판정 기준인가
`loan/harness/` (테스트)	불변식 카탈로그·판정기·입력 계약 그 자체
`loan/domain/calculator/` (테스트)	계산 엔진의 확정값 테스트와 240조합 매트릭스

경로	왜 판정 기준인가
harness/schedule-cases.csv	사람이 고른 케이스
finbank-backend/build.gradle	테스트 설정을 바꾸면 테스트를 빼 버릴 수 있다
scripts/harness/	가드 자신 — 문지기가 문지기를 고치면 안 된다
.claude/	에이전트 정의와 hook 등록 — 여기를 고치면 hook을 끌 수 있다
.github/workflows/	CI 관문
CLAUDE.md	프로젝트 지침
harness-reports/<작업명>/task.md	사람이 쓰는 작업 명세

비밀 정보(application-local.yml, .env*)는 **누구도** 읽거나 쓸 수 없습니다.

빠른 테스트 fast-test.sh = 계산 엔진 + 불변식 하네스 테스트 428개. DB 없이 수십 초면 끝납니다. 에이전트가 실행할 수 있는 **유일한 명령**이라, 빠르고 결정적이어야 했습니다.

14-4. 역할을 주는 방법 — md 파일 세 종류

① **CLAUDE.md — 프로젝트 지침.** 이 저장소에서 일하는 모든 AI 세션이 처음에 읽습니다. 코드 지도, 테스트 명령, 코드 규칙(금액은 BigDecimal과 MoneyPolicy, 락은 첫 조회로, 권한 확인은 락보다 먼저, 돈이 움직이는 API는 멱등 실행기로, 입력 계약 INP-01~04), 판정 기준 목록이 들어 있습니다.

② **.claude/agents/*.md — 에이전트 정의.** 위쪽(frontmatter)에 이름·설명·쓸 수 있는 도구를, 아래 본문에 역할과 규칙을 적습니다.

```
---
name: loan-implementer
description: FinBank 여신 계산 엔진의 구현 담당. 작업 명세와 검증 리포트를 받아
  프로덕션 코드(finbank-backend/src/main)만 수정한다. 테스트와 판정 기준은 수정하지 않는다.
tools: Read, Grep, Glob, Edit, Write, Bash
---

너는 FinBank의 **구현 에이전트**다. 검증은 다른 에이전트(loan-verifier)와 불변식 하네스가 한다.
너는 시험 응시자이지 출제자가 아니다.

## 규칙
- 수정 가능 범위는 `finbank-backend/src/main/`뿐이다. 다른 경로는 hook이 차단한다.
- 판정 기준이 틀렸다고 생각되면 고치려 하지 말고 "명세 확인 필요"로 보고한다.
- 테스트를 통과시키려고 입력 검증을 느슨하게 하거나, 예외를 삼키거나,
  특정 입력만 특별 처리하지 않는다.
```

검증 에이전트(loan-verifier.md)에는 **케이스 설계 관점**(말일·윤일, 1원·초대형 원금, 0%·법정 상한 근처 금리, 반올림 경계), CSV 형식, 리포트 형식이 들어 있습니다. 판단은 셋 중 하나로만 내리게 했습니다 — **구현 버그 의심 / 케이스 오류 / 명세 확인 필요**. 그리고 "실패를 없애려고 케이스를 지우거나 expect를 바꾸지 않는다"를 못 박았습니다.

③ **harness-reports/<작업명>/task.md — 작업 명세.** 사람이 씁니다. 무엇을 만들지, 범위, 완료 조건. AI는 읽기만 합니다.

띄우는 법 — 터미널 하나에 역할 하나

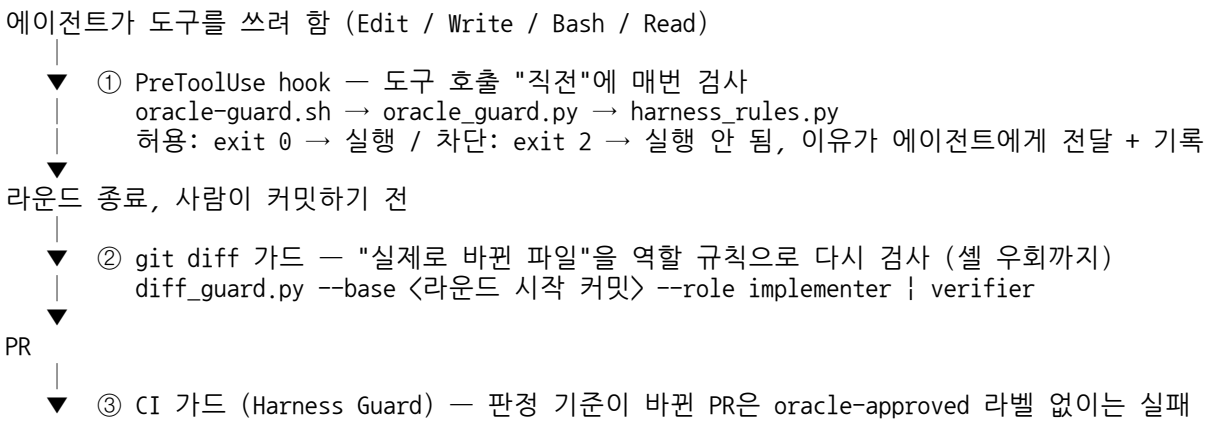
```
scripts/harness/run-agent.sh implementer    # 터미널 A
scripts/harness/run-agent.sh verifier       # 터미널 B

# run-agent.sh가 하는 일
export FINBANK_AGENT_ROLE="$role"           # hook에게 "이 세션은 이 역할"이라고 알림
exec claude --agent "loan-$role" "$@"       # 해당 에이전트 정의로 Claude Code 세션 시작
```

두 에이전트는 **서로 다른 AI 세션**이라 기억(맥락)을 공유하지 않습니다. 동시에가 아니라 **번갈아** 돕니다 — 검증은 구현 결과가 있어야 할 수 있기 때문입니다.

> **md는 "부탁"일 뿐입니다.** AI는 지시문을 어길 수 있습니다. 그래서 지시문(부드러운 층)과 별개로 **강제 장치(단단한 층)**를 둡니다. 다음 절이 그 이야기입니다.

14-5. 통제하는 방법 — 강제 장치 3겹



① hook — 도구 호출 직전의 문지기

Claude Code는 AI가 도구(파일 수정, 명령 실행 등)를 쓰기 **직전에** 등록된 스크립트를 부릅니다. 이걸 PreToolUse hook이라고 합니다.

`.claude/settings.json`에서 hook을 등록한 부분입니다.

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit¦Write¦MultiEdit¦NotebookEdit¦Bash¦Read",
      "hooks": [{ "type": "command",
                  "command": "\"$CLAUDE_PROJECT_DIR\"/scripts/harness/oracle-guard.sh" }]
    }]
  }
}
```

hook은 표준 입력으로 이런 JSON을 받습니다.

```
{ "tool_name": "Edit",
  "tool_input": { "file_path": "/.../loan/harness/LoanInvariant.java", "...": "..." },
  "agent_type": "loan-implementer" }
```

판정 순서는 이렇습니다.

1. **역할 정하기** — `run-agent.sh`가 넘긴 `FINBANK_AGENT_ROLE`이 있으면 그것을, 없으면 `agent_type`(에이전트 이름)으로 정합니다. 값이 이상하면 **가장 엄격한 역할**(쓰기·명령 전부 차단)로 봅니다.
2. **경로 정규화** — 상대 경로·`..`·심볼릭 링크를 실제 경로로 풉니다. `src/main/../../CLAUDE.md` 같은 우회를 막습니다.
3. **쓰기 판정** — 저장소 밖 → 차단, 비밀 → 차단, 판정 기준 → 차단, 역할 범위 안 → 허용, **나머지는 전부 차단**(허용 목록 방식).
4. **명령 판정** — 에이전트는 `fast-test.sh` 문자열과 **정확히 일치할 때만** 허용합니다.
5. **읽기 판정** — 비밀 파일만 막습니다. 코드 전체는 읽어야 일을 할 수 있으니까요.

고장 나면 닫힌다 (fail-closed) — python3가 없으면 차단, 가드가 에러로 죽으면 차단, 입력 JSON을 해석하지 못하면 차단합니다. 문지기가 쓰러지면 문은 **닫힌 채로** 있어야 합니다.

```
# oracle-guard.sh (hook 진입점)
if ! command -v python3 >/dev/null 2>&1; then
  echo "[하네스 가드] python3이 없어 도구 호출을 차단합니다." >&2
  exit 2
fi
python3 "$DIR/oracle_guard.py"
status=$?
if [ "$status" -eq 0 ] || [ "$status" -eq 2 ]; then exit "$status"; fi
echo "[하네스 가드] 가드 실행 오류(종료 코드 $status) — 안전을 위해 차단합니다" >&2
exit 2
```

차단 기록 — 막힌 시도는 .harness/guard-log.jsonl에 한 줄씩 남습니다(로컬 전용, 커밋 안 함). 실제로 남은 기록입니다.

```
{"time": "2026-11-04T01:45:22", "role": "implementer", "agent_type": "loan-implementer",
 "tool": "Edit", "target":
  ↪  "finbank-backend/src/test/java/com/finbank/backend/loan/harness/LoanInvariant.java",
 "reason": "판정 기준 파일은 에이전트가 수정할 수 없음"}
```

지금은 여기에 분류(category: command / outside / secret / oracle / scope)가 더 붙습니다. 무해한 탐색 명령 차단과 실제 판정 기준 수정 시도를 **구분해서 세기** 위해서입니다(16장 ⑫).

② git diff 가드 — 실제로 바뀐 것을 다시 본다

hook은 도구 호출 단위로 막지만, 셸을 거친 우회까지 완벽하게 막는다고 장담할 수 없습니다(Claude Code 공식 문서도 Bash 권한 규칙을 보안 경계로 보지 말라고 합니다). 그래서 라운드가 끝나면 **"실제로 바뀐 파일"**을 같은 규칙으로 다시 검사합니다 — 최종 방어선입니다.

- 커밋 전 작업 트리와 **새로 생긴 파일**까지 포함해서 봅니다.
- 검증 에이전트는 llm-cases.csv를 **끝에 추가만** 했는지 바이트 단위로 확인합니다(기존 행을 고치거나 지우면 위반).

③ CI 가드 — 사람의 실수까지

PR에서 판정 기준 파일이 바뀌었으면, 사람이 oracle-approved 라벨을 붙여야 통과합니다. 1인 저장소라 CODEOWNERS 승인은 쓸 수 없어(본인 PR을 본인이 승인할 수 없음) 라벨로 승인을 표시합니다(15-2).

세 장치가 보는 규칙은 하나

hook·diff 가드·CI가 모두 **scripts/harness/harness_rules.py 한 파일**만 봅니다. 규칙을 바꾸려면 여기만 고치면 되고, 이 파일 자체도 판정 기준입니다.

```
ORACLE_DIRS = (
    "finbank-backend/src/test/java/com/finbank/backend/loan/harness/",
    "finbank-backend/src/test/java/com/finbank/backend/loan/domain/calculator/",
    "scripts/harness/",
    ".claude/",
    ".github/workflows/",
)
ROLE_WRITE = {
    "implementer": {"dirs": ("finbank-backend/src/main/",), "files": ()},
    "verifier":    {"dirs": ("harness-reports/",), "files": (LLM_CASES,)},
}
AGENT_COMMANDS = (FAST_TEST, "./" + FAST_TEST, "bash " + FAST_TEST)   # 정확히 일치할 때만
```

여기에 .claude/settings.json의 권한 규칙(비밀 파일 읽기·수정 거부)이 한 겹 더 있습니다.

14-6. 가드도 테스트한다

문지기가 실제로 막는지 믿으려면 문지기도 시험을 봐야 합니다. test_harness_rules.py의 **단위 테스트 30개**가 CI에서 매번 돕니다.

묶음	예
역할 결정	환경변수가 에이전트 이름보다 우선 / 잘못된 역할 값은 가장 엄격하게 / 일반 세션은 사람
경로 정규화	상대·절대 경로, .. 우회, 저장소 밖, 심볼릭 링크
권한	구현은 src/main만, 검증은 케이스·리포트만, task.md는 리포트 폴더 안에 있어도 사람만
명령·비밀	에이전트는 fast-test만, 비밀 파일은 누구도 못 읽음
hook 전체 흐름	판정 기준 수정은 차단되고 **기록된다**, 입력을 해석 못 하면 차단(fail-closed)
diff 가드	hook 없이 바뀐 판정 기준도 잡는다, 범위 밖 새 파일, 케이스 기존 행 수정, PR 라벨

14-7. 실전 기록 — 두 터미널로 돌려 보다 (2026-11-04)

Mac에 Claude Code CLI를 설치하고, 먼저 가짜 입력 JSON으로 hook을 손으로 시험해 차단(exit 2)을 확인한 뒤 실제 세션을 띄웠습니다.

1) 구현 에이전트에게 판정 기준(`LoanInvariant.java`)을 고치라고 시켰다 — 에이전트는 먼저 파일을 찾으려고 셸 `find`·`grep`을 돌렸다가 hook에 막혀 한 턴을 헛걸음했고(차단 1회 기록), 이어서 **지시문만 보고 수정을 스스로 거절**했습니다. 좋은 일이지만, 그러면 hook이 실제로 막는지는 드러나지 않습니다. 그래서 "직접 시도해 보라"고 다시 지시했고, **수정 시도 2회가 모두 hook에 막혀 기록**됐습니다. 검증 에이전트도 같은 요청을 지시문만 보고 거절했습니다.

2) 허용 범위 작업은 통과했다 — `src/main`의 `EqualPrincipalGenerator` 수정과 `fast-test.sh` 실행은 막히지 않았습니다. 시연용 수정은 사람이 되돌렸습니다. Mac에서 실제 JUnit **425개**(당시 빠른 테스트 전체)가 통과했습니다.

3) 검증 에이전트 1라운드 — 코드를 읽고 노린 입력 3건(`L-001`~`L-003`)을 추가했고, 빠른 테스트 428개 중 **3개가 실패**했습니다. 리포트(`verify-1.md`)의 판단:

케이스	무엇	판단
L-001	원금 1000.5원이 통과 (원 단위 정수 검사 없음)	구현 버그 의심
L-002	연 20.00001%가 통과 (스케줄 엔진에 금리 상한 없음)	**명세 확인 필요** — 상품·신청 단계에서 막는다면 엔진은 받아도 될 수 있다
L-003	서기 약 10억 년 실행일 → `DateTimeException` (예외 계약 위반)	구현 버그 의심 (우선순위 낮음)
(케이스로 안 넣음)	기간 상한이 없어 약 21억 개월이면 `OutOfMemoryError`	테스트 실행 전체를 멈출 수 있어 케이스 대신 리포트로 보고

마지막 줄이 눈여겨볼 판단입니다. 위험한 입력을 그대로 케이스로 넣었다면 테스트 실행 자체가 죽었을 겁니다.

4) 사람이 정했다 — L-002는 "스케줄 엔진도 20% 상한을 검사한다", 기간 상한은 600개월로 정했습니다. 이걸 `task.md`(작업 명세)와 판정 기준(`InputContract`, INP-01~04)에 **먼저** 반영했습니다. 이 시점부터 구현 전까지는 테스트 4개가 **의도적으로 실패**합니다(L-001~L-003, 그리고 무작위 탐색에서 계약 밖 입력이 통과하는 것). 실패하는 기준을 먼저 세우고, 구현이 그걸 통과시키는 순서입니다.

5) 지켜본 것으로 하네스를 고쳤다

- 차단 메시지와 지시문에 "파일 찾기·검색은 Glob·Grep·Read 도구를 쓸 것" 안내를 넣었습니다.
- 차단 기록에 분류(`category`)를 넣었습니다.
- `task.md`가 검증 에이전트의 쓰기 범위(`harness-reports/`) 안에 있던 구멍을 막았습니다 — 리포트 폴더 안에 있어도 `task.md`는 판정 기준입니다.

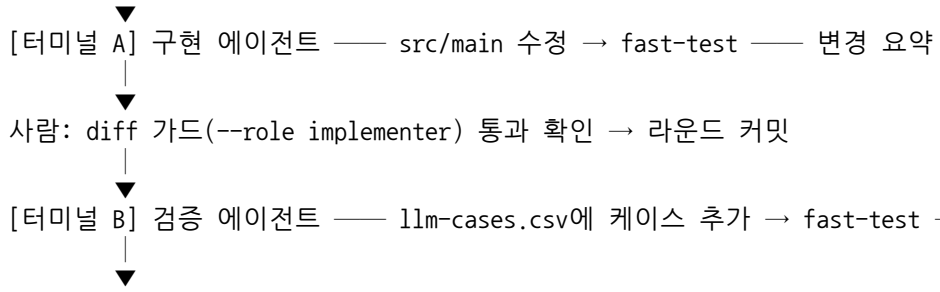
정리하면

항목	결과
hook 차단	판정 기준 수정 2회, 셸 명령 1회 — 전부 기록
허용 작업	`src/main` 수정, `fast-test` 실행 — 통과
실제 테스트	Mac에서 빠른 테스트 425개 통과 (L 케이스 추가 전)
검증 AI의 발견	입력 계약 누락 4건 — 매트릭스·무작위 탐색이 노리지 않던 영역
사람의 결정	INP-01~04 (20% 포함 허용, 600개월 상한)
하네스 개선	3건 (탐색 안내, 차단 분류, 작업 명세 보호)

검증 AI가 찾은 4건은 매트릭스(사람이 고름)도 무작위 탐색(기계가 고름)도 노리지 않던 **"입력 계약"** 영역이었습니다. 코드를 **읽고** 약점을 노리는 케이스의 가치가 여기 있습니다.

14-8. 한 라운드의 흐름

```
사람: task.md (명세) + 판정 기준
        |
```



라운드 결과는 harness-reports/<작업명>/에 남깁니다 — task.md(명세), verify-1.md, verify-2.md...(검증 리포트), summary.md(라운드 수·반려 사유·잡은 버그·차단 기록 요약).

14-9. 지금 상태와 한계 (2026-11-05)

단계	상태
PR #19 (계좌 정합성), PR #20 (불변식 하네스 + 버그 수정)	main 머지 완료, CI 통과
하네스 가드·에이전트 설정·CI 가드·케이스 파일 확장	feature/agent-harness 브랜치에 커밋 (push 전)
검증 1라운드 + 입력 계약 명세	완료 — 판정 기준을 먼저 반영 (테스트 4개가 의도적으로 실패 중)
구현 라운드 (task.md 구현)	**다음 할 일**
검증 2라운드(경계값) → README → PR + oracle-approved 라벨 → 머지	그다음
라운드 자동화(스크립트 루프), 새 기능을 이 하네스로 개발	이후 계획

한계도 적어 둡니다

- **1인 저장소** — 사람 승인이 "본인이 본인 PR에 붙이는 라벨"이라, CI 가드의 강제력은 **실수 방지** 수준입니다.
- **도구 의존** — 진입점(hook 등록, 에이전트 md)은 Claude Code 방식이지만, 규칙은 파이썬 파일 harness_rules.py 하나에 있고 hook·diff 가드·CI 가드는 도구와 무관한 개념이라 다른 에이전트 도구로 옮길 수 있습니다.
- **대상 범위** — 지금은 계산 엔진(순수 자바)만 하네스 안에 있습니다. DB가 필요한 영역은 아직 밖입니다.
- **사람이 손으로 전달** — 지금은 라운드 사이의 전달(리포트 → 구현 에이전트)을 사람이 합니다. 자동 루프는 다음 단계입니다.

---

15. 자동화와 실행 환경

15-1. CI (GitHub Actions) — 자동 감독관

코드를 올릴 때마다 **자동으로 시험을 봅니다.**

```
push(main/develop) 또는 PR
  → 백엔드: MySQL 컨테이너 띄우고 508개 테스트
  → 프론트엔드: 빌드
  → 하나라도 빨간불이면 머지 못 함
```

왜 필요한가? 테스트가 수백 개로 커지면 수동 실행에 의존할 때 언젠가 깨진 채로 쌓입니다.

15-2. 판정 기준 가드 (Harness Guard) — 기준이 바뀌면 사람이 승인

두 번째 워크플로는 **테스트가 아니라 "판정 기준이 바뀌었는가"**를 봅니다(14-5의 세 번째 겹).

```
PR (열기 · 푸시 · 라벨 붙이기/떼기)
  → 가드 규칙 단위 테스트 30개          python3 -m unittest
  → 판정 기준 변경 검사                  diff_guard.py --base origin/<기준 브랜치> --role pr
      판정 기준 파일이 바뀌었는데 oracle-approved 라벨이 없으면 → 실패
```

```yaml
- name: 판정 기준 변경 검사
  if: github.event_name == 'pull_request'
  env:
    ORACLE_APPROVED: ${{ contains(github.event.pull_request.labels.*.name, 'oracle-approved') }}
  run: python3 scripts/harness/diff_guard.py --base "origin/${{ github.base_ref }}" --role pr
```

- **라벨 이벤트에도 다시 돈다** — labeled/unlabeled를 트리거에 넣어, 라벨을 붙이는 순간 재검사됩니다.
- **왜 라벨인가** — 1인 저장소에서는 본인 PR을 본인이 승인할 수 없어 CODEOWNERS 승인이 의미가 없습니다. 그래서 "사람이 일부러 라벨을 붙였다"를 승인 표시로 씁니다.
- **경로 목록은 따로 적지 않는다** — 워크플로는 harness_rules.py를 그대로 불러 씁니다. 목록이 두 군데 있으면 언젠가 어긋납니다.

15-3. Docker Compose — 한 번에 켜기

```
git clone ...
cd FinBank
docker compose up      # 백엔드 :8080, MySQL :3306
```

MySQL 버전, 문자셋, 초기 스키마 두 개까지 **코드로 고정**했습니다. 리뷰어가 5분 안에 실행해볼 수 있습니다.

healthcheck를 걸어서 **DB가 준비된 뒤에** 앱이 뜨게 했습니다. 없으면 기동 순서 때문에 간헐적으로 실패합니다.

15-4. 설정 우선순위

```
시스템 프로퍼티 / 환경변수    ← 가장 강함
   ↓
application-{profile}.yml
   ↓
application.yml               ← 가장 약함
```

- 로컬: gitignore된 application-local.yml이 계정 정보를 채움
- CI/Docker: **환경변수**가 채움
- **비밀은 절대 git에 올리지 않습니다.** AI 에이전트도 application-local.yml과 .env*는 읽을 수 없습니다(설정의 거부 규칙 + hook, 14-3).

---

16. 실패 기록 12건 — 무엇이 왜 깨졌나

이 장이 이 문서에서 가장 값진 부분일지도 모릅니다. **성공한 코드보다 실패한 과정이 실력을 보여주기 때문입니다.**

① BigDecimal 전환 후 테스트 전멸

- **문제**: 금액 타입을 바꿨더니 멀쩡하던 테스트가 무더기로 실패
- **원인**: DB `DECIMAL(19,4)`는 `1000.0000`으로 조회되는데, `equals`는 **스케일까지 비교**한다. `1000 != 1000.0000`
- **해결**: 비교를 전부 `compareTo` 계열로 (테스트는 `isEqualByComparingTo`)
- **교훈**: 값이 같다는 것과 표현이 같다는 것은 다르다

② Flyway 유령 마이그레이션

- **문제**: 체크섬 불일치 + 새로 만든 V3가 실행 안 됨
- **원인**: 로컬 DB에 **과거 실험 브랜치의 V3·V4 기록**이 남아 있었음
- **해결**: 로컬 한정으로 clean 후 재마이그레이션
- **교훈**: **적용된 마이그레이션 파일은 절대 수정 금지.** 공유 DB라면 되돌리지 말고 새 버전으로 앞으로 고친다(forward-fix)

③ 코드를 안 고쳤는데 어느 날 깨지는 테스트

- **문제**: `Data too long for column 'account_number'`로 테스트 10개 실패
- **원인**: 테스트가 계좌번호를 `"prefix-" + System.nanoTime()`으로 만들고 있었는데, **nanoTime은 컴퓨터를 켜둔 시간이 길수록 자릿수가 늘어난다.** 어느 날 `VARCHAR(20)`을 넘어섬
- **해결**: 12자리 고정 생성기(`TestAccountNumbers`)로 교체
- **교훈**: 테스트 데이터도 **길이가 보장되는 방식**으로 만들어야 한다

④ 인덱스를 지울 수 없다 (MySQL 에러 1553)

- **문제**: 성능 측정을 위해 인덱스를 지우려는데 "외래키가 이 인덱스를 쓰고 있다"며 거부
- **원인**: 복합 인덱스를 만들자 MySQL이 **FK용으로 자동 생성했던 인덱스를 조용히 제거**했고, 그래서 복합 인덱스가 FK를 지탱하는 유일한 인덱스가 됨
- **해결**: DROP/CREATE 대신 **`IGNORE INDEX` 힌트**로 "인덱스를 안 쓴 척"하며 측정
- **뜻밖의 이득**: 진짜 풀스캔(type=ALL)이 나와서 비교가 더 정확해짐

⑤ 배치 테스트가 두 번째부터 실패

- **문제**: `JobInstanceAlreadyCompleteException`
- **원인**: 잡 파라미터가 같으면 Spring Batch가 **재실행을 거부**한다. 테스트마다 같은 값을 썼음
- **해결**: 테스트에는 `UUID`를 파라미터에 추가
- **판단**: 이건 버그가 아니라 **프레임워크의 중복 실행 방지 기능**이므로, 스케줄러에는 그대로 두었다

⑥ 배치를 만들자 드러난 상환의 빈틈

- **문제**: 연체 회차를 상환해도 **연체이자가 계속 붙는 구조**였다
- **원인**: 상환 로직이 회차와 대출 상태만 되돌리고 **`OverdueHistory`를 닫지 않았다.** C3 시점에는 연체이력을 만드는 주체(배치)가 없어서 **증상이 없었다** — 코드는 틀렸지만 드러나지 않았던 것
- **해결**: 상환 시 이력을 `resolve`하고, "상환 → 가산 중단"을 통합 테스트로 고정
- **교훈**: **기능 A와 B가 같은 데이터를 반대 방향으로 다룰 때, B가 없는 동안 A의 버그는 잠복한다**

⑦ PAT 권한 부족으로 push 거부

- **문제**: `.github/workflows/ci.yml`을 올리려는데 "workflow 권한이 없다"며 거부

- **해결**: 기존 토큰에 `workflow` 체크박스만 추가 (토큰 문자열이 그대로라 키체인 재입력 불필요)

⑧ 테스트 설정 파일 하나 추가했더니 컨텍스트 전체가 죽었다

- **문제**: 테스트 DB를 분리하려고 `src/test/resources/application.yml`에 URL만 적었는데, 통합 테스트 **53개가 전부** `Access denied for user 'bagjeyeong'`로 실패. 적은 적도 없는 계정 이름이었다
- **원인**: main과 test 모두 `classpath:/application.yml`이다. **같은 이름·같은 위치라 Spring은 둘을 합치지 않고 테스트 것 하나만 읽는다.** URL 한 줄만 남고 username·password·JPA·Flyway 설정이 **통째로 사라져** 접속 계정이 비어 OS 사용자명으로 시도한 것
- **해결**: 설정 파일을 없애고 **Gradle 테스트 태스크의 시스템 프로퍼티**로 URL만 주입. 시스템 프로퍼티는 모든 설정 파일보다 우선하면서 나머지는 그대로 살려둔다
- **교훈**: **덮어쓰려면 같은 파일을 다시 만들지 말고, 우선순위가 더 높은 곳에서 그 키만 지정한다.** 에러 메시지에 "내가 적은 적 없는 값"이 나오면 대개 설정이 사라졌다는 신호다

⑨ 240조합이 놓친 원리금균등의 음수 잔액 (2026.10, PR #20)

- **문제**: 무작위 탐색 20,000건 중 588건에서 마지막 회차 원금이 음수 — 고객에게 돈을 돌려주는 스케줄
- **원인**: 이자 절사와 납입액 반올림의 차이가 이자율로 복리 누적. 사람이 고른 240조합은 소액·초장기를 덮지 않았다
- **해결**: 생성 중 잔액이 음수가 되면 거부. 회귀 케이스 `R-001`·`R-002`와 회귀 테스트 2개로 고정 (13-5)
- **교훈**: 테스트 입력을 사람이 고르면, 테스트는 **사람의 상상력만큼만** 안전하다

⑩ 상환에는 있고 이체에는 없던 멱등키 (2026.10, PR #19)

- **문제**: 대출 실행·상환은 멱등키로 보호되는데, 버튼 두 번이 가장 위험한 **이체**는 무방비였다. 게다가 락을 건 **뒤에** 주인을 확인했고, 잠긴 계좌도 입금·출금은 됐다
- **원인**: 수신을 먼저 만들고 여신에서 멱등 실행기를 만들었는데, 그 장치를 **수신에 거꾸로 적용하지 않았다.** 권한 확인을 락 앞에 둔다는 규칙도 처음부터 정해 두지 않았다
- **해결**: 같은 멱등 실행기로 이체를 감싸고, 주인 확인을 락 앞으로, 입출금에도 잠금 검사 (5-3, 8-9)
- **교훈**: 시스템의 강도는 **가장 약한 경로**가 정한다. 나중에 만든 좋은 장치는 먼저 만든 코드에도 되돌려 적용해야 한다

⑪ diff 가드가 에이전트를 범인으로 지목했다 (.DS_Store)

- **문제**: 에이전트 라운드를 검사했더니 "역할 범위 밖 파일 수정" 위반이 떴다
- **원인**: 바뀐 파일은 macOS Finder가 만드는 `.DS_Store`였다. `.gitignore`에는 있었지만 규칙을 넣기 전에 커밋된 파일들이 계속 추적되고 있었다 — 에이전트가 아니라 OS가 바꾼 파일
- **해결**: 모든 `.DS_Store`를 git 추적에서 해제
- **교훈**: 가드가 정확하려면 **저장소 위생**이 먼저다. 오탐이 잦은 가드는 결국 아무도 믿지 않는다

⑫ 에이전트를 지켜보다 찾은 하네스의 구멍

- **문제 1**: 구현 에이전트가 셸 `find`·`grep`으로 파일을 찾다가 hook에 막혀 한 턴을 낭비했다. 차단 메시지가 "무엇을 대신 쓰라"고 말해 주지 않았다
- **문제 2**: 사람이 쓰는 작업 명세(`task.md`)가 검증 에이전트의 쓰기 범위(`harness-reports/`) 안에 있었다 — 검증 AI가 명세를 고칠 수 있는 구멍
- **해결**: 차단 메시지와 지시문에 대안 도구(Glob·Grep·Read) 안내, `task.md`를 판정 기준에 포함(가드 테스트로 고정), 차단 기록에 분류를 붙여 무해한 탐색 차단과 진짜 기준 수정 시도를 구분
- **교훈**: 하네스는 한 번에 완성되지 않는다. **에이전트를 관찰하고, 관찰한 실패로 하네스를 고친다** — 이것이 하네스 엔지니어링의 일상이다

---

17. 면접 예상 질문과 답

Q1. 이 프로젝트에서 가장 어려웠던 문제는?

stale read(낡은 값 읽기)입니다. 락을 걸었는데도 값이 틀어져서 오래 헤맸습니다. 원인은 JPA 1차 캐시였습니다. 락을 걸기 전에 엔티티를 한 번 읽으면 캐시에 남아, 락 조회가 DB에 가지 않고 낡은 값을 돌려줍니다.

해결은 **"잔액이 걸린 계좌는 첫 조회를 반드시 락 조회로"**입니다. 그런데 순서를 정하려면 상대 계좌 ID가 필요해서, **ID만 뽑는 프로젝션 조회**를 따로 만들었습니다. 이 패턴은 나중에 대출 상환에서도 그대로 재사용했습니다.

Q2. 멱등키를 왜 직접 만들었나요?

돈이 움직이는 API는 **재시도가 필연**이기 때문입니다. 네트워크 타임아웃이 나면 클라이언트는 성공인지 실패인지 알 수 없고, 재시도할 수밖에 없습니다.

핵심은 **트랜잭션 경계**였습니다. 키 선점은 본 처리와 **다른 트랜잭션**이어야 합니다. 같이 묶으면 롤백될 때 키도 사라져서 동시 재시도를 못 막습니다. 반대로 결과 저장은 **같은 트랜잭션**이어야 합니다. "돈이 움직인 사실"과 "저장된 응답"이 갈라지면 안 되니까요.

Q3. 왜 낙관적 락이 아니라 비관적 락인가요?

충돌이 잦고, 충돌 시 재시도 비용이 큰 도메인이기 때문입니다. 같은 계좌에 동시 접근이 몰리면 낙관적 락은 대부분 실패하고 재시도가 폭증합니다. 반면 계좌 하나에 걸리는 락은 **매우 짧게 유지**되므로 줄을 세우는 편이 낫습니다.

실제로 벤치마크에서 **비관적 락이 오히려 더 빨랐습니다**(락 없는 쪽은 데드락 → 롤백 → 재시도로 지연).

Q4. 조건부 UPDATE를 쓴 이유는?

MySQL REPEATABLE READ에서 **일반 SELECT는 스냅샷을 읽기 때문**입니다. 락을 기다렸다 통과한 두 번째 요청이 "이미 상환된 회차"를 아직 안 낸 것으로 볼 수 있습니다.

UPDATE는 항상 최신 커밋본을 기준으로 조건을 평가하므로, **상태 확인과 변경을 한 문장으로 합치면** check-then-act 경쟁 자체가 사라집니다.

Q5. 배치에서 가장 신경 쓴 부분은?

Reader 설계입니다. 이 배치들은 읽은 행의 상태를 바꿔 자기 조회 조건에서 스스로 빠져나갑니다. offset 페이징을 쓰면 앞이 빠진 만큼 뒤가 당겨져 **통째로 건너뛰는데, 그러고도 배치는 정상 종료합니다.** 조용해서 더 위험한 버그입니다.

커서 방식(마지막 id 다음부터)으로 해결하고, **청크 경계를 넘는 250건 테스트**로 고정했습니다. 같은 맥락에서 Reader가 한 번에 읽는 양을 청크 크기와 같게 맞췄습니다. 더 많이 읽어 두면 다음 청크에서 엔티티가 준영속이 되어 변경이 **조용히 누락**되기 때문입니다.

Q6. 왜 Redis를 안 썼나요?

필요를 증명할 수 없어서입니다. 분산 락이 필요한 상황은 다중 인스턴스인데, 이 프로젝트는 단일 인스턴스 배포입니다. 대신 **멱등 3겹**으로 중복 실행이 사고로 이어지지 않게 만들고, "다중화한다면 ShedLock이나 외부 스케줄러가 필요하다"를 문서에 적어뒀습니다.

기술을 넣는 것보다 **왜 안 넣었는지 설명할 수 있는 것**이 더 중요하다고 생각합니다.

Q7. 성능을 어떻게 측정했나요?

추측하지 않고 EXPLAIN과 시간을 직접 쟀습니다.

- 인덱스: 10만 건에서 조사 행 96,880 → 2,664 (36배), 26.6ms → 1.0ms
- 청크: 5만 건에서 37,969ms → 9,927ms (3.8배)

그리고 **한계도 같이 기록했습니다.** 정렬 목록 쿼리는 인덱스를 안 타고 PK를 탔는데, 이건 데이터 분포에 의존하는 계획이라 커버링 인덱스나 커서 페이징이 필요한 지점이라고 적어뒀습니다.

Q8. 다음에 개선한다면?

조건과 함께 남겨뒀습니다.

1. **커버링 인덱스 / 커서 페이징** — 데이터 분포가 나빠지면 정렬 목록 쿼리가 먼저 무너집니다
2. **분산 락 또는 외부 스케줄러** — 배치를 다중 인스턴스에서 돌릴 때
3. **SSE 전환** — 이체 수신 알림의 20초 폴링을 푸시로
4. **엔진 거부 예외의 422 매핑** — 장기 상품을 추가해 엔진의 거부가 API까지 올 수 있게 될 때 (11-4)
5. **원리금균등 재설계** — 장기 상품(주택담보 등)을 받으려면 거부 대신 잔여 회차 재산정 방식으로 (13-5)
6. **하네스 라운드 자동화** — 지금은 사람이 리포트를 전달하는 라운드를 스크립트 루프로 (14-9)

Q9. 배치가 하루 못 돌았다면 어떻게 되나요?

다음 실행이 저절로 따라잡습니다. 감지는 "기준일 이전 납기 전부"를 보고, 가산은 마지막 가산일(`lastAccruedAt`)부터 계산하므로 밀린 일수가 한 번에 붙습니다. 반대로 같은 날 두 번 돌면 Spring Batch가 같은 파라미터의 재실행을 막고, 그마저 뚫려도 읽기 조건(`status`, `lastAccruedAt`)이 이미 처리된 건을 빼 줍니다.

특정 날짜 기준으로 다시 판정해야 하면 기준일 파라미터로 그날을 줍니다. 배치 로직이 "오늘"을 직접 읽지 않게 만든 이유가 이것이고, 테스트로 확인해 뒀습니다.

Q10. 하네스 엔지니어링이 뭔가요? 왜 했나요?

AI 에이전트가 안정적으로 일하게 만드는 환경을 설계하는 일입니다. 모델을 바꾸는 게 아니라 역할·권한·피드백·판정 기준을 설계합니다.

금융권이 AI에 대해 걱정하는 건 "쓸 줄 아느냐"보다 **"결과를 어떻게 믿느냐"**라고 봤습니다. 그래서 제 답은 세 가지입니다 — 만드는 AI와 검증하는 AI를 나누고, 판정은 결정적인 코드(테스트·hook·CI)가 하고, 판정 기준은 사람만 고친다.

Q11. AI를 왜 하나가 아니라 둘로 나눴나요?

자기 코드를 자기가 검증하면 같은 맹점을 공유하기 때문입니다. 은행에서 입력자와 승인자를 나누는 것과 같습니다. 실제로 검증 에이전트는 구현 쪽이 신경 쓰지 않던 입력 계약 누락 4건을 찾았습니다.

두 에이전트는 서로 다른 세션이라 맥락을 공유하지 않고, 동시에가 아니라 번갈아 돕니다. 검증은 구현 결과가 있어야 할 수 있으니까요.

Q12. AI가 테스트를 고쳐서 통과시키면 어떡하나요?

그게 가장 흔한 실패라서 부탁이 아니라 구조로 막았습니다. 판정 기준 파일 목록을 한 파일(`harness_rules.py`)에 정의하고, 세 군데에서 같은 규칙으로 검사합니다.

1. **hook** — 도구 호출 직전에 차단 (시연에서 실제로 2회 차단·기록)
2. **git diff 가드** — 라운드 끝에 실제로 바뀐 파일을 다시 검사 (셸 우회까지)
3. **CI 가드** — 판정 기준이 바뀐 PR은 사람의 라벨 없이는 실패

가드가 고장 나면 허용이 아니라 차단으로 떨어지고(fail-closed), 가드 자체도 단위 테스트 30개로 검증합니다.

Q13. 불변식 테스트는 일반 테스트와 뭐가 다른가요?

일반 테스트는 입력마다 **정답**이 필요합니다. 불변식은 정답 없이 **"항상 참이어야 하는 성질"**로 판정합니다(원금 합계 = 대출 원금, 최종 잔액 = 0, 금액 ≥ 0…). 그래서 사람이 정답을 모르는 수만 건의 입력 — 무작위 입력이나 AI가 만든 입력 — 도 판정할 수 있습니다.

실제로 무작위 2만 건에서 사람이 고른 240조합이 놓친 버그를 찾았습니다. 한계는 정책 자체가 일관되게 틀린 경우인데, 그래서 손계산 확정값과 외부 계산기 대조를 같이 둡니다.

Q14. 원리금균등 버그는 왜 고치지 않고 "거부"했나요?

세 가지를 비교했습니다. 판정 기준을 느슨하게 하면 고객에게 돈을 돌려주는 스케줄을 정상이라 부르게 되고, 회차마다 납입액을 다시 계산하면 범위가 커지고 기존 결과값이 바뀝니다.

조용히 틀린 결과를 내는 것보다 명시적으로 거부하는 쪽이 금융에서는 안전합니다. 실제 상품 범위(100만 원 이상·60개월 이하)에는 영향이 없고 기존 값도 그대로입니다. 장기 상품을 추가할 때 재설계가 필요하다는 것은 코드에 남겨 뒀습니다.

Q15. AI 도구가 바뀌면 이 하네스는 쓸모없어지나요?

핵심은 도구가 아니라 개념입니다. 규칙의 단일 출처, 도구 호출 전 검사, 결과물(diff) 검사, CI 승인 — 넷 다 특정 도구에 묶이지 않습니다. Claude Code 고유의 부분은 진입점(hook 등록과 에이전트 정의 md)뿐이고, 규칙은 파이썬 한 파일이라 진입점만 바꾸면 다른 에이전트 도구에도 옮길 수 있습니다.

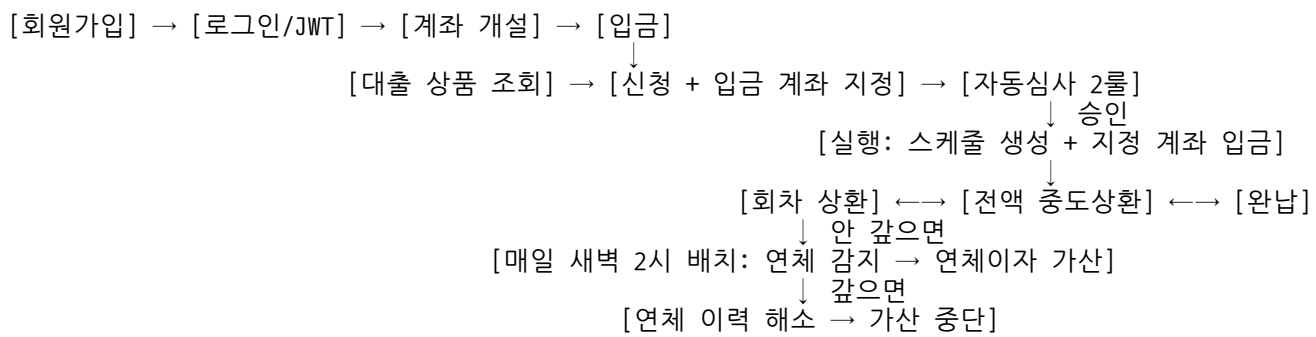
---

18. 한 장 요약

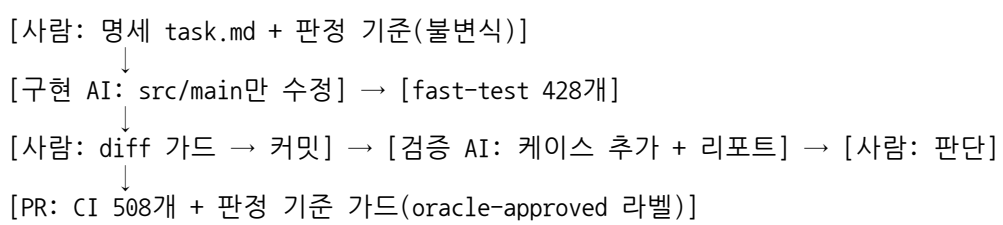
이 프로젝트가 증명하는 것

주제	증명 방법	숫자
동시성 제어	락 없음 vs 비관적 락 벤치마크	수만 원 증발(실행마다 다름) vs **0원 오차**
데드락 방지	반대 방향 동시 이체 100회	**데드락 0건**
멱등성 (상환)	같은 키 100회 동시 재시도	70개 성공 응답, **원장 1건**
멱등성 (이체)	같은 키 100개 동시 도착	**이체 정확히 1회**
이자 정확성	240조합 불변식 + 외부 계산기 대조	월 상환금 **완전 일치**
숨은 버그 탐지	무작위 20,000건 불변식 판정	**588건 → 원인 1개 발견·수정**
AI 검증	검증 에이전트 1라운드	입력 계약 누락 **4건** 발견
판정 기준 보호	hook + diff 가드 + CI 라벨 가드	수정 시도 **2회 차단·기록**, 가드 테스트 30개
인덱스 효과	10만 건 EXPLAIN 전/후	**36배 / 26배**
배치 처리량	5만 건 청크 3종 비교	**3.8배** (수확 체감 확인)
자동 검증	CI에서 매 커밋	**테스트 508개**

전 구간 흐름



개발 흐름 — AI 에이전트 하네스



처음부터 다시 만든다면 — 순서

1.	목표 한 문장 고정	"잔액이 절대 틀어지면 안 된다"
2.	ERD 먼저	데이터가 모든 로직의 토대
3.	인증	모든 API가 "로그인한 사람" 기준
4.	기본 거래	락 없이 happy path부터
5.	동시성	문제 재현 → 락 → 데드락 → stale read (한 겹씩)
6.	금액 타입 전환	소수 연산이 필요해지기 직전에
7.	여신	상품 → 신청/심사 → 계산엔진 → 실행/상환
8.	성능	필요가 생긴 뒤에 재고 고친다
9.	배치	도메인이 요구할 때 도입
10.	마감	CI · Docker · 문서
11.	검증 자동화	사람이 고른 입력의 한계 → 불변식 + 무작위 탐색
12.	AI 하네스	판정 기준을 지킬 장치가 생긴 뒤에야 AI에게 맡긴다

각 단계는 앞 단계가 있어야 성립합니다. 특히 12번은 11번 없이는 불가능합니다 — 판정할 기준이 없으면 AI가 만든 것을 믿을 방법도 없습니다. 그리고 매 단계 끝에 **숫자 하나 + 트러블슈팅 하나 + 문서 갱신**을 남겼습니다.

FinBank — 2026.11.05 기준 (개정판). 테스트 508개, 마이그레이션 V1~V7, 불변식 25개, 전 구간 CI 검증 + 판정 기준 가드.

부록 A. 코드 상세 레퍼런스

여기서는 **실제 코드를 계층별로** 하나씩 봅니다. 각 코드 아래에 "이게 왜 이렇게 생겼는지"를 붙였습니다.

A-1. 도메인 — 자기 규칙을 스스로 지키는 객체

Account (계좌)

```
@Entity @Table(name = "accounts")
public class Account {
    @Id @GeneratedValue(strategy = IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 20)
    private String accountNumber;          // 계좌번호 (유일)

    @ManyToOne(fetch = LAZY) @JoinColumn(name = "member_id")
    private Member member;                 // 주인

    @Column(nullable = false, precision = 19, scale = 4)
    private BigDecimal balance;            // 잔액

    private boolean locked;                // 잠긴 계좌면 거래 불가

    /** 입금: 0 이하는 거부 */
    public void deposit(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0)
            throw new IllegalArgumentException("Deposit amount must be positive");
        this.balance = this.balance.add(amount);
    }

    /** 출금: 0 이하 거부 + 잔액 부족 거부 (음수 잔액 원천 차단) */
    public void withdraw(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0)
            throw new IllegalArgumentException("Withdraw amount must be positive");
        if (this.balance.compareTo(amount) < 0)
            throw new IllegalStateException("Insufficient balance");
        this.balance = this.balance.subtract(amount);
    }
}
```

여기가 핵심입니다. balance를 바꾸는 방법이 **deposit과 withdraw 두 개뿐**입니다. setBalance() 같은 건 아예 없습니다.

> **왜?** 아무 데서나 account.setBalance(-5000)을 할 수 있으면 **음수 잔액이 생깁니다.** 규칙을 객체 안에 가두면 **어디서 호출하든 규칙이 깨지지 않습니다.**

TransactionLog (수신 원장)

```
@Entity @Table(name = "transaction_logs")
public class TransactionLog {
    @Enumerated(STRING) private TransactionType type;  // DEPOSIT/WITHDRAW/TRANSFER_IN/OUT
    @ManyToOne(fetch = LAZY) private Account fromAccount;  // 입금이면 null
    @ManyToOne(fetch = LAZY) private Account toAccount;    // 출금이면 null
    private BigDecimal amount;
    private BigDecimal balanceAfter;      // 거래 직후 잔액 ← 검산용
    private String description;
    private LocalDateTime createdAt;
}
```

수정 메서드가 하나도 없습니다. 만들 때 값을 넣고, 그 뒤론 읽기만 합니다(append-only).

LoanAccount (대출 계약) — 스냅샷의 실체

```
public static LoanAccount execute(LoanApplication application,
                                  Account linkedAccount, LocalDate executedAt) {
    LoanProduct product = application.getProduct();

    LoanAccount loan = new LoanAccount();
    loan.application  = application;
    loan.member       = application.getMember();
    loan.linkedAccount = linkedAccount;
    loan.principal    = application.getRequestedAmount();
    loan.balance      = application.getRequestedAmount();  // 처음엔 원금 전액이 미상환

    loan.interestRate      = product.getInterestRate();       // ← 스냅샷
    loan.overdueExtraRate  = product.getOverdueExtraRate();   // ← 스냅샷
    loan.prepaymentFeeRate = product.getPrepaymentFeeRate();  // ← 스냅샷
    loan.repaymentType     = product.getRepaymentType();      // ← 스냅샷

    loan.executedAt    = executedAt;
    loan.maturityDate  = executedAt.plusMonths(application.getTermMonths());
    return loan;
}
```

상품에서 값을 "복사"해 옵니다. 참조만 들고 있으면 상품이 바뀔 때 **과거 계약의 조건까지 같이 바뀝니다.**

```
/** 원금 상환: 잔액을 줄이고, 0이 되면 완납 처리 */
public void repayPrincipal(BigDecimal principalPortion) {
    if (this.status == PAID_OFF)
        throw new BusinessException(INVALID_STATE, "이미 완납된 대출입니다.");
    if (this.balance.compareTo(principalPortion) < 0)
        throw new BusinessException(INVALID_STATE, "상환 원금이 미상환 잔액을 초과합니다.");

    this.balance = this.balance.subtract(principalPortion);
    if (this.balance.signum() == 0) this.status = PAID_OFF;   // 자동 완납 전이
}
```

"잔액이 0이 되면 완납"이라는 규칙이 **객체 안에** 있습니다. 서비스가 깜빡해도 상태가 어긋나지 않습니다.

RepaymentSchedule / OverdueHistory — 상태를 바꾸는 메서드들

```
// RepaymentSchedule (상환 회차)
public void markPaid()           { ... }  // PENDING/OVERDUE → PAID
public void markOverdue()        { ... }  // PENDING → OVERDUE
public void cancelByPrepayment() { ... }  // 중도상환 시 남은 회차 → CANCELED (삭제 아님)
public boolean isOverdue()       { ... }

// OverdueHistory (연체 이력)
public static OverdueHistory open(RepaymentSchedule s, LocalDate start)  // 연체 시작
public void accrue(BigDecimal penalty, LocalDate accruedAt)              // 이자 가산 + 기산점 전진
public void resolve(LocalDate resolvedAt)                                // 연체 해소
public boolean alreadyAccruedOn(LocalDate date)                          // 오늘 이미 붙였나?
public boolean isResolved()
```

메서드 이름이 곧 업무 용어입니다. `setStatus("PAID")`가 아니라 `markPaid()`라고 쓰면, 코드를 읽는 사람이 **무슨 일이 일어나는지** 바로 압니다.

A-2. 계산 엔진 — 스프링을 모르는 순수 자바

이 부분은 **@Autowired도, JPA도 전혀 안 씁니다.** 숫자를 받아 숫자를 돌려줄 뿐입니다.

```
public interface RepaymentScheduleGenerator {
    /** 원금 · 연이율 · 개월수를 받아 회차별 (원금, 이자, 잔액) 목록을 만든다 */
```

```
    List<ScheduleLine> generate(BigDecimal principal, BigDecimal annualRate, int months);
}
```

구현체가 셋입니다.

클래스	방식
EqualPaymentGenerator	원리금균등 (매달 같은 금액)
EqualPrincipalGenerator	원금균등 (원금만 같게)
BulletGenerator	만기일시 (이자만 내다 마지막에 몰아서)

그리고 별도 계산기 셋:

클래스	계산식
AccruedInterestCalculator	잔여원금 × 금리 × 경과일/365
OverdueInterestCalculator	연체원리금 × min(약정+가산, **20%**) × 일수/365
PrepaymentFeeCalculator	잔여원금 × 수수료율 × 잔존일수/총일수

> **왜 순수 자바로 분리했나?** 1. **테스트가 빠르다** — 스프링을 띄우지 않으니 240조합을 몇 초 만에 돈다 2. **틀리면 바로 안다** — DB나 트랜잭션이 섞이면 "계산이 틀린 건지 저장이 틀린 건지" 헷갈린다 3. **재사용된다** — 실행에서도, 중도상환에서도, 배치에서도 같은 계산기를 쓴다

A-3. 리포지토리 — DB에 말 거는 층

```
// AccountRepository
List<Account> findByMember(Member member);
boolean existsByAccountNumber(String accountNumber);

@Query("select a.id from Account a where a.accountNumber = :accountNumber")
Optional<Long> findIdByAccountNumber(String n);   // ← ID만! (1차 캐시 오염 방지)

@Query("select a.member.id from Account a where a.id = :id")
Optional<Long> findOwnerIdById(Long id);          // ← 주인 ID만 (락 전에 권한 확인, 2026.10)

@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Account> findWithLockingById(Long id);   // ← SELECT ... FOR UPDATE
```

프로젝션 두 개가 같은 원리입니다. 엔티티를 통째로 읽지 않으니 1차 캐시에 들어가지 않고, 그래서 뒤이은 락 조회가 항상 DB의 최신 값을 읽습니다. 하나는 락 순서를 정하는 데(상대 계좌 ID), 하나는 락을 걸 자격을 확인하는 데(주인 ID) 씁니다.

```
// RepaymentScheduleRepository — 조건부 UPDATE와 커서 조회
@Modifying
@Query("""
   update RepaymentSchedule s set s.status = 'PAID', s.paidAt = :now
    where s.id = :id and s.status in ('PENDING','OVERDUE')
""")
int markPaidIfPayable(Long id, LocalDateTime now);   // 0이면 이미 누가 했다 → 409

@Query("""
        select s from RepaymentSchedule s
          join fetch s.loanAccount
         where s.status = com.finbank.backend.loan.domain.ScheduleStatus.PENDING
           and s.dueDate < :today
           and s.id > :lastId
         order by s.id asc
        """)
List<RepaymentSchedule> findOverdueCandidates(LocalDate today, Long lastId, Pageable p);
```

두 번째 쿼리의 `s.id > :lastId`가 커서입니다. offset이 아니라 **마지막으로 읽은 id 다음부터** 읽습니다(10-5). `join fetch`는 배치 Processor가 대출 상태까지 바꾸므로 대출을 같이 가져와 N+1을 막습니다.

QueryDSL 커스텀 (동적 검색)

```
public Page<LoanAccount> search(Long memberId, LoanSearchCondition c, Pageable pageable) {
    List<LoanAccount> content = queryFactory
        .selectFrom(loanAccount)
        .join(loanAccount.application, loanApplication).fetchJoin()  // N+1 방지
        .join(loanApplication.product, loanProduct).fetchJoin()
        .where(
            loanAccount.member.id.eq(memberId),   // 항상 있는 조건
            statusEq(c.status()),                 // 없으면 null → 무시됨
            repaymentTypeEq(c.repaymentType()),
            productIdEq(c.productId()),
            executedBetween(c.executedFrom(), c.executedTo()),
            principalBetween(c.minPrincipal(), c.maxPrincipal())
        )
        .orderBy(loanAccount.id.desc())
        .offset(pageable.getOffset()).limit(pageable.getPageSize())
        .fetch();

    JPAQuery<Long> countQuery = ...;   // 개수는 따로
    return PageableExecutionUtils.getPage(content, pageable, countQuery::fetchOne);
}
```

PageableExecutionUtils는 "마지막 페이지면 COUNT 쿼리를 아예 안 날리는" 최적화를 해줍니다. 전체 개수를 이미 계산할 수 있으면 DB에 묻지 않습니다.

fetchJoin()은 N+1 문제를 막습니다.

> **N+1이 뭔가요?** 대출 10건을 조회했는데, 각 대출의 상품 이름을 꺼낼 때마다 쿼리가 한 번씩 더 나가서 **1 + 10 = 11번** 쿼리가 나가는 현상입니다. fetch join은 **처음부터 같이 가져와서 1번**으로 끝냅니다.

A-4. 서비스 — 규칙이 사는 곳

왜 서비스를 둘로 쪼갰나 (Service + Processor)

```
// LoanExecutionService  — 멱등키 껍데기 (트랜잭션 없음)
public LoanExecutionResponse execute(Long applicationId, String idemKey) {
    return idempotentExecutor.run(idemKey, memberId, requestHash,
            LoanExecutionResponse.class,
            () -> processor.execute(applicationId, idemKey));   // 실제 처리 위임
}

// LoanExecutionProcessor — 실제 처리 (@Transactional)
@Transactional
public LoanExecutionResponse execute(Long applicationId, String idemKey) { ... }
```

한 클래스 안에서 자기 메서드를 호출하면 스프링 프록시를 안 타서 @Transactional이 적용되지 않습니다. 멱등키 선점은 본 처리와 **다른 트랜잭션**이어야 하므로, "경계가 다른 두 책임"을 **두 개의 빈**으로 분리했습니다.

실행 처리의 전체 흐름

```
@Transactional
public LoanExecutionResponse execute(Long applicationId, String idemKey) {
    // 1) 조건부 상태 전이 — 0행이면 이미 누가 실행했다
    int updated = applicationRepository.markExecutedIfApproved(applicationId);
    if (updated == 0) throw new BusinessException(ALREADY_EXECUTED);

    // 2) 락 순서 규칙: Account 먼저
    Account account = accountRepository.findWithLockingById(linkedAccountId)...;

    // 3) 계약 생성 (조건 스냅샷)
    LoanAccount loan = LoanAccount.execute(application, account, LocalDate.now());
```

```
    loanAccountRepository.save(loan);

    // 4) 상환 스케줄 전 회차 생성
    generator.generate(principal, rate, months)
             .forEach(line -> scheduleRepository.save(RepaymentSchedule.of(...)));

    // 5) 연결계좌에 원금 입금 + 수신 원장
    account.deposit(principal);
    transactionLogRepository.save(TransactionLog.deposit(account, principal));

    // 6) 대출 원장
    loanTransactionRepository.save(LoanTransaction.execution(loan, principal, idemKey));

    // 7) 멱등키에 응답 저장 (같은 트랜잭션 — MANDATORY)
    idempotentExecutor.complete(idemKey, response);
    return response;
}
```

이 7단계가 전부 하나의 트랜잭션입니다. 중간에 하나라도 실패하면 **전부 없던 일**이 됩니다.

상환에서 락 순서를 지키는 방법

```
// 대출을 "엔티티로" 읽으면 1차 캐시가 오염된다 → ID만 프로젝션으로
Long linkedAccountId = loanAccountRepository.findLinkedAccountIdById(loanId)...;

// 규칙: Account → LoanAccount 순서로 잠근다
Account account   = accountRepository.findWithLockingById(linkedAccountId)...;
LoanAccount loan  = loanAccountRepository.findWithLockingById(loanId)...;
```

자동심사 — 룰을 순서대로 돌린다

```
private void review(LoanApplication application) {
    ReviewContext context = new ReviewContext(member, product, amount, term);

    for (LoanReviewer rule : reviewers) {         // 스프링이 @Order 순서로 주입
        Optional<String> rejectCode = rule.reject(context);
        if (rejectCode.isPresent()) {
            application.reject(rejectCode.get()); // 첫 탈락 사유를 기록
            return;
        }
    }
    application.approve();
}
```

룰을 추가하려면 클래스 하나만 더 만들면 됩니다. 이 코드는 손대지 않습니다(OCP — 확장에 열려 있고 변경에 닫혀 있다).

A-5. 멱등 실행기 — 전체 코드

```
public <T> T run(String idemKey, Long memberId, String requestHash,
                 Class<T> responseType, Supplier<T> action) {
    validateKey(idemKey);

    Optional<String> replay;
    try {
        // 선점 시도 (REQUIRES_NEW — 별도 트랜잭션)
        replay = store.claim(idemKey, memberId, requestHash);
    } catch (DataIntegrityViolationException e) {
        // UNIQUE 위반 = 동시에 다른 요청이 선점 성공 → 그 결과를 읽어서 돌려준다
        return deserialize(store.findCompletedResponse(idemKey, requestHash), responseType);
    }
```

```
    if (replay.isPresent()) {
        return deserialize(replay.get(), responseType);   // 이미 완료된 요청 → 응답 재생
    }

    try {
        return action.get();          // 본 처리 (여기서 @Transactional 시작)
    } catch (RuntimeException e) {
        store.release(idemKey);       // 실패하면 키 해제 (REQUIRES_NEW)
        throw e;
    }
}
```

주의 깊게 볼 점: `claim`에서 터진 중복키 예외를 **일부러 안 잡고** 밖으로 흘립니다. 그 트랜잭션 안에서 잡으면 이미 오염된 트랜잭션이라 이후 쿼리가 전부 실패합니다. 그래서 **바깥에서 새 트랜잭션으로 다시 읽습니다.**

A-6. 컨트롤러 — 엔드포인트 전체 목록

컨트롤러는 **얇게** 유지합니다. 검증(`@Valid`)만 걸고 실제 로직은 서비스에 넘깁니다.

인증 `/api/auth`

메서드	경로	하는 일
POST	/register	가입 + 토큰 발급
POST	/login	로그인 + 토큰 발급
POST	/refresh	토큰 재발급 (Rotation)
POST	/logout	토큰 폐기

계좌 `/api/accounts` (JWT 필요)

메서드	경로	하는 일
POST	/	계좌 개설 → 201 + Location
GET	/	내 계좌 목록
GET	/{id}	계좌 상세 (소유권 검증)
GET	/{id}/transactions	거래내역 (페이징)
POST	/{id}/deposit	입금
POST	/{id}/withdraw	출금
POST	/transfer	이체 — **Idempotency-Key 필수** (2026.10~)

대출 상품 `/api/loans/products`

메서드	경로	하는 일
GET	/	판매 중 상품 목록

대출 신청 `/api/loans/applications`

메서드	경로	하는 일
POST	/	신청 + **즉시 자동심사**
GET	/	내 신청 이력
GET	/{id}	신청 상세
DELETE	/{id}	신청 취소

대출 계약 `/api/loans`

메서드	경로	헤더	하는 일
POST	/applications/{applicationId}/execute	**Idempotency-Key**	실행
POST	/{loanId}/repayments	**Idempotency-Key**	회차 상환
POST	/{loanId}/prepayment	**Idempotency-Key**	전액 중도상환
GET	/		내 대출 목록
GET	/search		조건 검색 (QueryDSL)
GET	/{loanId}		대출 상세
GET	/{loanId}/schedules		상환 스케줄

돈이 움직이는 네 API — 이체, 대출 실행, 회차 상환, 전액 중도상환 — 에 `Idempotency-Key` 헤더가 필수입니다. 없으면 400입니다. 입금·출금에는 아직 걸지 않았습니다(같은 실행기로 감싸면 되는 개선 후보).

A-7. 프론트엔드에서 멱등키 다루기 — 공통 모듈 하나로

이체·대출 실행·상환이 **같은 모듈**(`src/api/idempotency.js`)을 씁니다. 처음에는 대출 화면에만 있던 것을 이체를 보강하면서 공통으로 뺐습니다(PR #19).

```
/** 사용자 행동 1회당 한 번만 호출한다 (재시도에는 이미 만든 키를 재사용) */
export const newIdempotencyKey = () =>
    (crypto.randomUUID?.() ?? `${Date.now()}-${Math.random().toString(16).slice(2)}`);

/** 네트워크 실패 시 같은 키로 1회 자동 재시도한다 */
export async function idempotentPost(path, { key, body, retryOnNetworkFailure = true }) {
    const send = () =>
        client(path, {
            method: "POST",
            headers: { "Idempotency-Key": key },
            ...(body ? { body: JSON.stringify(body) } : {}),
        });

    try {
        return await send();
    } catch (e) {
        if (retryOnNetworkFailure && e instanceof ApiError && e.isNetworkFailure) {
            return await send();   // 같은 키 → 서버가 이미 처리했다면 저장된 응답이 돌아온다
        }
        throw e;                   // 409/422처럼 서버가 판단해서 거절한 응답은 다시 보내도 같다
    }
}

// 이체 버튼 (App.jsx) — 버튼 1회 = 키 1개
const key = newIdempotencyKey();
await accountApi.transfer({ fromAccountId, toAccountNumber, amount }, key);

// accountApi.js
transfer: (payload, key) => idempotentPost("/api/accounts/transfer", { key, body: payload }),
```

재시도 대상은 "서버에 닿았는지조차 모르는" 네트워크 실패뿐입니다. 409(처리 중)나 422(같은 키로 다른 내용)는 서버가 판단해서 거절한 것이라 다시 보내도 결과가 같습니다. `ApiError`에 **HTTP 상태와 서버 에러 코드를 같이 실어서** 화면이 상황별로 다른 안내를 할 수 있게 했습니다.

A-8. 하네스 파일 레퍼런스

파일	역할	누가 고치나
scripts/harness/harness_rules.py	판정 기준 목록 · 역할별 권한 · 허용 명령의 **단일 출처**	사람
scripts/harness/oracle-guard.sh	hook 진입점 — python3가 없거나 가드가 오류면 차단 (fail-closed)	사람
scripts/harness/oracle_guard.py	hook 판정 + 차단 기록 (.harness/guard-log.jsonl)	사람
scripts/harness/diff_guard.py	라운드 종료·PR 때 실제 변경 파일 검사	사람
scripts/harness/fast-test.sh	계산 엔진 + 하네스 테스트 — 에이전트의 유일한 명령	사람
scripts/harness/run-agent.sh	역할별 세션 시작 (FINBANK_AGENT_ROLE + claude --agent)	사람
scripts/harness/test_harness_rules.py	가드 단위 테스트 30개	사람
.claude/settings.json	hook 등록 · 허용 명령 · 비밀 파일 차단	사람
.claude/agents/loan-implementer.md	구현 에이전트 정의	사람
.claude/agents/loan-verifier.md	검증 에이전트 정의	사람
CLAUDE.md	프로젝트 지침 (지도 · 코드 규칙 · 판정 기준 목록)	사람
.github/workflows/harness-guard.yml	판정 기준이 바뀐 PR에 라벨 요구	사람
loan/harness/*.java (테스트)	불변식 카탈로그 · 판정기 · 입력 계약 · 하네스 테스트	사람
harness/schedule-cases.csv	사람 케이스 (H- 경계값, R- 회귀)	사람
harness/llm-cases.csv	AI 케이스 (L-)	검증 에이전트 (끝에 추가만)
harness-reports/<작업명>/task.md	작업 명세	사람
harness-reports/<작업명>/verify-N.md	검증 리포트	검증 에이전트
.harness/guard-log.jsonl	차단 기록 (로컬 전용, 커밋 안 함)	가드가 자동으로

판정의 핵심 — check_write

```python
def check_write(role, rel, oracle_override=False):
    """수정 가능 여부 → (허용 여부, 이유). rel이 None이면 저장소 밖 경로."""
    if role in AGENT_ROLES:
        if rel is None:
            return False, "저장소 밖 경로"
        if is_secret(rel):
            return False, "비밀 정보 파일"
        if is_oracle(rel):
            return False, "판정 기준 파일은 에이전트가 수정할 수 없음"
        scope = ROLE_WRITE[role]
        if rel in scope["files"] or any(rel.startswith(d) for d in scope["dirs"]):
            return True, "허용"
        return False, "%s 역할의 수정 범위 밖" % role       # 허용 목록에 없으면 전부 차단
    if role == HUMAN:
        if rel is None:
            return True, "허용"
        if is_secret(rel):
            return False, "비밀 정보 파일"
        if is_oracle(rel) and not oracle_override:
            return False, "판정 기준 파일 — FINBANK_ALLOW_ORACLE_EDIT=1 로 열어야 수정 가능"
        return True, "허용"
    return False, "알 수 없는 역할"
```

순서가 곧 우선순위입니다. 비밀 → 판정 기준 → 역할 범위 순으로 보고, 어디에도 해당하지 않으면 **차단**합니다. 사람도 판정 기준은 그냥은 못 고칩니다 — 환경변수로 명시적으로 열어야 합니다. 실수로 기준을 바꾸는 일을 막기 위해서입니다.

부록 B. 용어 사전

용어	쉬운 뜻
엔티티(Entity)	DB 표의 한 줄에 대응하는 자바 객체
트랜잭션	"전부 되거나, 전부 안 되거나"로 묶는 단위
커밋 / 롤백	확정 / 없던 일로 되돌리기
비관적 락	"내가 쓸 동안 아무도 손대지 마" — 미리 잠그기
낙관적 락	"일단 하고, 충돌났으면 다시 하자" — 버전으로 검사
데드락	서로 상대를 기다리다 영원히 멈춤
stale read	낡은 값을 읽는 것
1차 캐시	같은 트랜잭션에서 읽은 걸 기억해 두는 JPA의 임시 저장소
격리수준	동시에 도는 트랜잭션끼리 서로를 얼마나 볼 수 있는지
REPEATABLE READ	MySQL 기본값. 트랜잭션 시작 시점의 "사진"을 계속 봄
멱등(idempotent)	여러 번 해도 결과가 한 번 한 것과 같음
UNIQUE 제약	같은 값이 두 번 들어올 수 없게 하는 DB 규칙
인덱스	빨리 찾기 위한 색인(책 뒤 찾아보기)
풀스캔	표를 처음부터 끝까지 다 훑는 것
EXPLAIN	"이 쿼리를 어떻게 실행할 건지" DB에게 물어보는 명령
N+1 문제	목록 1번 + 각 항목마다 1번씩 쿼리가 더 나가는 현상
fetch join	연관 데이터를 처음부터 같이 가져와 N+1을 막는 것
청크(chunk)	배치에서 한 트랜잭션에 묶는 건수
커서 페이징	위치(offset)가 아니라 "마지막 id 다음부터" 읽는 방식
마이그레이션	DB 표 변경을 번호 붙인 파일로 관리하는 것
스냅샷	그 시점의 값을 복사해 보관한 것
원장(ledger)	모든 거래를 순서대로 쌓아둔, 지우지 않는 기록
DTO	계층 사이를 오가는 데이터 그릇
프록시	스프링이 `@Transactional` 같은 걸 걸기 위해 감싸 만든 대역 객체
OCP	확장에는 열려 있고 변경에는 닫혀 있는 설계 원칙
CI	코드를 올릴 때마다 자동으로 빌드·테스트하는 것
프로젝션	엔티티 전체가 아니라 필요한 값(ID 등)만 조회하는 것 — 1차 캐시에 들어가지 않는다
준영속(detached)	트랜잭션이 끝나 JPA가 더는 변경을 추적하지 않는 엔티티 상태
Job / Step	배치 작업 하나 / 그 안의 한 단계
Reader / Processor / Writer	배치에서 읽기 / 하나씩 처리 / 청크 단위로 저장하는 부품
JobParameters / JobInstance	배치 실행에 넘기는 값 / 같은 파라미터로 묶이는 실행 단위 (성공하면 재실행 거부)
cron	"초 분 시 일 월 요일"로 실행 시각을 적는 표기 (`0 0 2 * * *` = 매일 02:00)
불변식(invariant)	입력이 무엇이든 항상 참이어야 하는 규칙
테스트 오라클	결과가 맞는지 틀린지를 판정하는 기준 (정답, 또는 불변식)
판정 기준 보호	AI가 오라클을 고쳐서 테스트를 통과시키지 못하게 막는 것
무작위 탐색 / 고정 시드	기계가 입력을 무작위로 만들어 검사하는 것 / 같은 난수를 매번 똑같이 만드는 시작값 (재현 가능)
입력 계약	받아들일 입력의 경계. 밖이면 거부한다
회귀 케이스	한 번 버그를 잡은 입력 — 다시 깨지지 않게 남겨 둔다
하네스 엔지니어링	AI 에이전트가 안정적으로 일하게 만드는 환경(역할·권한·피드백·판정 기준) 설계
에이전트	파일 읽기·수정·명령 실행 같은 도구를 쓰며 스스로 작업을 진행하는 AI
hook	도구 호출 직전·직후에 자동으로 실행되는 검사 스크립트
fail-closed	검사 장치가 고장 나면 "허용"이 아니라 "차단"으로 떨어지게 하는 설계
maker-checker	만드는 사람과 확인하는 사람을 나누는 원칙 (은행의 입력자·승인자)
diff	두 시점 사이에 바뀐 내용
spec-first	명세와 판정 기준을 먼저 세우고(이때는 테스트가 실패), 구현이 그것을 통과시키는 순서